

Guia de reciclaje de componentes web

Funcionalidades 41 a 50 - cierre del recurso unificado

HTML + CSS + JavaScript modular y version normal

Este documento continua el mismo formato de las guias anteriores. Aqui se cubren las funcionalidades restantes del recurso unificado, desde Todo list hasta Responsive design. La meta es que puedas reciclar cada funcionalidad sin depender del proyecto original completo.

El orden sigue la tabla de contenido del recurso unificado: 41 Todo list, 42 Drag and drop, 43 Conversor C/F/K, 44 Cambio de fondo, 45 Imprimir pagina, 46 Reset visual, 47 Helpers DOM/storage, 48 Version modular, 49 Version normal y 50 Responsive design.

Rango	Funcionalidades cubiertas	Objetivo de esta guia
41 a 50	Todo list, Drag and drop, Conversor C/F/K, Cambio de fondo, Imprimir pagina, Reset visual, Helpers DOM/storage, Version modular, Version normal y Responsive design.	Cerrar el banco de componentes con funcionalidades de productividad, estado visual, organizacion de JavaScript y adaptabilidad responsive.

Tabla de contenido

- 41. Todo list reutilizable
- 42. Drag and drop para reordenar tareas
- 43. Conversor C/F/K con botones de unidad
- 44. Cambio de fondo segun boton seleccionado
- 45. Imprimir pagina o seccion
- 46. Reset visual de estados y clases
- 47. Helpers DOM y storage
- 48. Version modular con export/import
- 49. Version normal sin modulos
- 50. Responsive design reutilizable

Como usar esta guia

- 1 Lee primero la descripcion de la funcionalidad y confirma que es lo que necesitas.
- 2 Copia el HTML minimo y cambia textos, ids y data-attributes.
- 3 Copia el CSS y adapta colores, espaciados, responsive y estados visuales.
- 4 Escoge una sola version de JavaScript: modular o normal.
- 5 Cambia los valores indicados en la tabla de variables de cada funcionalidad.
- 6 Prueba el checklist antes de reutilizar el componente en otro proyecto.

Regla importante: estas funcionalidades se pueden combinar. Por ejemplo, Todo list + Drag and drop + localStorage te permite crear una lista reordenable persistente. Conversor C/F/K + Cambio de fondo + Reset visual crea una mini app interactiva con estados visuales.

Convenciones que se repiten en las funcionalidades 41 a 50

Convencion	Para que sirve	Como adaptarla
storageKey	Clave unica para guardar datos en localStorage.	Usa nombres especificos como project-todos o sierra-notes para evitar mezclar datos.
data-id / data-unit / data-bg-class	Atributos HTML que comunican datos a JavaScript.	Cambia los valores, pero manten el mismo nombre si el JS los lee.
is-active / is-done / is-dragging	Clases de estado que activan estilos.	No las uses como clases base; JavaScript debe agregarlas o quitarlas.
callback	Funcion que se ejecuta cuando ocurre algo.	Usala para guardar cambios, mostrar toasts o conectar componentes.
@media	Reglas CSS para adaptar la vista a pantallas pequenas.	Ajusta breakpoints segun el diseno real.

Consejo de reutilizacion: cuando un componente usa ids, no repitas el mismo id mas de una vez. Si quieres dos TodoList en la misma pagina, cada una debe tener formId, inputId, listId y storageKey diferentes.

Mapa de combinaciones recomendadas

Combinacion	Para que sirve	Componentes involucrados
Todo list + Drag and drop	Lista de tareas reordenable con prioridades.	41 + 42 + 47
Conversor + Fondo dinamico + Reset	Calculadora interactiva con estados visuales y boton de reinicio.	43 + 44 + 46
Imprimir + Responsive	Reporte que se ve bien en pantalla y tambien en PDF/impresion.	45 + 50
Helpers + Version modular	Codigo mas limpio y reutilizable en proyectos grandes.	47 + 48
Version normal + Responsive	Proyecto simple sin modulos que aun se adapta a celular.	49 + 50

41. Todo list reutilizable

Aspecto	Explicacion
Que hace	Lista de tareas que permite crear, marcar como hecha y eliminar elementos. Es una base muy usada para paneles, checklists, tareas administrativas y prototipos CRUD.
Donde se usa en industria	Tableros de tareas, dashboards internos, seguimiento de pendientes, listas de compras, listas de validacion, checklists de onboarding y mini sistemas CRUD.
Como se personaliza	Cambia el id del formulario, input, lista, clave de localStorage, estructura de cada tarea y textos visibles. Puedes agregar prioridad, categoria, fecha limite o responsable.

Mapa rapido de reciclaje

- 1 Ubica el lugar exacto del HTML donde quieres insertar el componente.
- 2 Copia el HTML minimo y cambia textos visibles, ids y data-attributes.
- 3 Copia el CSS y ajusta colores, separaciones, radios, sombras y responsive si hace falta.
- 4 Escoge JS modular o JS normal, no ambos al mismo tiempo.
- 5 Inicializa el componente con el id correcto y prueba la interaccion principal.

HTML minimo que debes copiar

index.html

```
<section class="panel todo-widget" id="todoWidget">
  <h2>Lista de tareas</h2>

  <form class="todo-form" id="todoForm">
    <input id="todoInput" type="text" placeholder="Escribe una tarea" required>
    <button class="btn" type="submit">Agregar</button>
  </form>

  <ul class="todo-list" id="todoList"></ul>
</section>
```

CSS del componente

styles.css

```

.todo-widget {
  display: grid;
  gap: 16px;
}
.todo-form {
  display: grid;
  grid-template-columns: 1fr auto;
  gap: 10px;
}
.todo-form input {
  min-height: 44px;
  padding: 10px 12px;
  border-radius: 14px;
  border: 1px solid rgba(148, 163, 184, .5);
}
.todo-list {
  display: grid;
  gap: 10px;
  margin: 0;
  padding: 0;
  list-style: none;
}
.todo-item {
  display: grid;
  grid-template-columns: auto 1fr auto;
  align-items: center;
  gap: 10px;
  padding: 12px;
  border-radius: 16px;
  background: rgba(255,255,255,.12);
  border: 1px solid rgba(255,255,255,.18);
}
.todo-item.is-done .todo-text {
  text-decoration: line-through;
  opacity: .55;
}
.todo-delete {
  border: 0;
  border-radius: 999px;
  padding: 8px 10px;
  cursor: pointer;
}

```

JavaScript modular con export/import

Usa esta version si tu HTML carga `<script type="module" src="script.js"></script>`. La clase o funcion vive en funciones.js y la instancia se hace en script.js.

funciones.js + script.js

```

export class TodoList {
  constructor({ formId, inputId, listId, storageKey = 'kit-todos' }) {
    this.form = document.getElementById(formId);
    this.input = document.getElementById(inputId);
    this.list = document.getElementById(listId);
    this.storageKey = storageKey;
    this.todos = JSON.parse(localStorage.getItem(this.storageKey)) || [];
    this.init();
  }

  init() {
    this.form.addEventListener('submit', (event) => {
      event.preventDefault();
      this.add(this.input.value);
      this.input.value = '';
    });
    this.render();
  }

  save() {
    localStorage.setItem(this.storageKey, JSON.stringify(this.todos));
  }

  add(text) {
    const cleanText = text.trim();
    if (!cleanText) return;
    this.todos.push({ id: crypto.randomUUID(), text: cleanText, done: false });
    this.save();
    this.render();
  }

  toggle(id) {
    this.todos = this.todos.map((todo) => todo.id === id ? { ...todo, done: !todo.done } : todo);
    this.save();
    this.render();
  }

  remove(id) {
    this.todos = this.todos.filter((todo) => todo.id !== id);
    this.save();
    this.render();
  }

  render() {
    this.list.innerHTML = this.todos.map((todo) => `
      <li class="todo-item ${todo.done ? 'is-done' : ''}">
        <input type="checkbox" ${todo.done ? 'checked' : ''} data-toggle-todo="${todo.id}">
        <span class="todo-text">${todo.text}</span>
        <button class="todo-delete" type="button" data-delete-todo="${todo.id}">Eliminar</button>
      </li>
    `).join('');

    this.list.querySelectorAll('[data-toggle-todo]').forEach((checkbox) => {
      checkbox.addEventListener('change', () => this.toggle(checkbox.dataset.toggleTodo));
    });
    this.list.querySelectorAll('[data-delete-todo]').forEach((button) => {
      button.addEventListener('click', () => this.remove(button.dataset.deleteTodo));
    });
  }
}

// script.js
import { TodoList } from './funciones.js';
new TodoList({ formId: 'todoForm', inputId: 'todoInput', listId: 'todoList', storageKey: 'mis-tareas' });

```

JavaScript normal sin modulos

Usa esta version si prefieres tener todo en un solo archivo como script-normal.js y cargarlo con `<script src="script-normal.js"></script>`.

script-normal.js

```

class TodoList {
  constructor(config) {
    this.form = document.getElementById(config.formId);
    this.input = document.getElementById(config.inputId);
    this.list = document.getElementById(config.listId);
    this.storageKey = config.storageKey || 'kit-todos';
    this.todos = JSON.parse(localStorage.getItem(this.storageKey)) || [];
    this.init();
  }
  init() {
    this.form.addEventListener('submit', (event) => {
      event.preventDefault();
      this.add(this.input.value);
      this.input.value = '';
    });
    this.render();
  }
  save() { localStorage.setItem(this.storageKey, JSON.stringify(this.todos)); }
  add(text) {
    const cleanText = text.trim();
    if (!cleanText) return;
    this.todos.push({ id: Date.now().toString(), text: cleanText, done: false });
    this.save();
    this.render();
  }
  toggle(id) {
    this.todos = this.todos.map((todo) => todo.id === id ? { ...todo, done: !todo.done } : todo);
    this.save();
    this.render();
  }
  remove(id) {
    this.todos = this.todos.filter((todo) => todo.id !== id);
    this.save();
    this.render();
  }
  render() {
    this.list.innerHTML = this.todos.map((todo) => `
      <li class="todo-item ${todo.done ? 'is-done' : ''}">
        <input type="checkbox" ${todo.done ? 'checked' : ''} data-toggle-todo="${todo.id}">
        <span class="todo-text">${todo.text}</span>
        <button class="todo-delete" type="button" data-delete-todo="${todo.id}">Eliminar</button>
      </li>
    `).join('');
    this.list.querySelectorAll('[data-toggle-todo]').forEach((el) => el.addEventListener('change', (
    ) => this.toggle(el.dataset.toggleTodo)));
    this.list.querySelectorAll('[data-delete-todo]').forEach((el) => el.addEventListener('click', (
    ) => this.remove(el.dataset.deleteTodo)));
  }
}
new TodoList({ formId: 'todoForm', inputId: 'todoInput', listId: 'todoList', storageKey: 'mis-tareas' });

```

Variables, clases, ids y atributos que debes cambiar

Elemento	Para que sirve	Que debes cambiar
todoForm	Formulario que captura la nueva tarea.	Cambia el id si tienes varios formularios.
todoInput	Input donde el usuario escribe.	Debe coincidir con el input real.
todoList	Contenedor donde se pintan las tareas.	Puede ser ul, div o section.
storageKey	Clave usada en localStorage.	Usa una clave unica por proyecto.
todo-item / is-done	Clases CSS de tarjeta y estado completado.	Cambia nombres si tu CSS usa otra convencion.

Implementacion paso a paso

- 1 Copia el HTML del widget donde quieras mostrar la lista.
- 2 Copia el CSS de la lista y ajusta colores o bordes segun tu diseno.
- 3 Copia la clase TodoList en funciones.js o en script-normal.js.
- 4 Inicializa la clase pasando los ids correctos.
- 5 Prueba agregar, marcar y eliminar una tarea.

6 Recarga la pagina para verificar que localStorage conserva los datos.

Errores comunes y como solucionarlos

- No se agrega la tarea: revisa que el formId y inputId sean correctos.
- Se duplican eventos: no inicialices TodoList dos veces con el mismo contenedor.
- Las tareas se pierden al recargar: revisa storageKey y permisos del navegador.
- El texto queda inseguro si aceptas HTML del usuario: en produccion conviene escapar el texto antes de pintar.

Checklist de prueba antes de reutilizar

- El componente existe en HTML y el id coincide con el usado en JavaScript.
- No hay ids duplicados en la pagina.
- La clase visual de estado se agrega y se quita correctamente.
- No aparecen errores en la consola del navegador.
- La funcionalidad sigue funcionando en pantalla pequena.
- El componente no rompe otros elementos del proyecto.

Mejoras opcionales

- Conectar el componente con un Toast para avisar acciones exitosas o errores.
- Guardar preferencias o datos en localStorage cuando tenga sentido.
- Agregar validaciones antes de ejecutar acciones importantes.
- Mejorar accesibilidad con aria-label, aria-live, focus-visible y navegacion por teclado cuando aplique.

42. Drag and drop para reordenar tareas

Aspecto	Explicacion
Que hace	Permite arrastrar elementos para cambiar su orden. Es muy util cuando el usuario necesita priorizar, ordenar columnas, mover cards o reorganizar listas.
Donde se usa en industria	Kanban, listas de tareas, gestores de prioridades, administradores de menus, orden de productos, dashboards y constructores visuales.
Como se personaliza	Cambia el selector de items, el contenedor, la clase visual al arrastrar y la funcion que guarda el nuevo orden. Si ya tienes TodoList, este componente se conecta al array de tareas.

Mapa rapido de reciclaje

- 1 Ubica el lugar exacto del HTML donde quieres insertar el componente.
- 2 Copia el HTML minimo y cambia textos visibles, ids y data-attributes.
- 3 Copia el CSS y ajusta colores, separaciones, radios, sombras y responsive si hace falta.
- 4 Escoge JS modular o JS normal, no ambos al mismo tiempo.
- 5 Inicializa el componente con el id correcto y prueba la interaccion principal.

HTML minimo que debes copiar

index.html

```
<ul class="sortable-list" id="sortableList">
  <li class="sortable-item" draggable="true" data-id="1">Diseñar navbar</li>
  <li class="sortable-item" draggable="true" data-id="2">Crear sidebar</li>
  <li class="sortable-item" draggable="true" data-id="3">Validar formulario</li>
</ul>
```

CSS del componente

styles.css

```
.sortable-list {
  display: grid;
  gap: 10px;
  margin: 0;
  padding: 0;
  list-style: none;
}
.sortable-item {
  padding: 14px 16px;
  border-radius: 16px;
  background: rgba(255,255,255,.12);
  border: 1px solid rgba(255,255,255,.18);
  cursor: grab;
  user-select: none;
  transition: transform .2s ease, opacity .2s ease, border-color .2s ease;
}
.sortable-item:active {
  cursor: grabbing;
}
.sortable-item.is-dragging {
  opacity: .45;
  transform: scale(.98);
  border-color: #60a5fa;
}
.sortable-item.drag-over {
  outline: 2px dashed #60a5fa;
  outline-offset: 4px;
}
```

JavaScript modular con export/import

Usa esta version si tu HTML carga `<script type="module" src="script.js"></script>`. La clase o funcion vive en funciones.js y la instancia se hace en script.js.

funciones.js + script.js

```
export class DragSorter {
  constructor(listId, onChange = null) {
    this.list = document.getElementById(listId);
    this.onChange = onChange;
    this.dragging = null;
    this.init();
  }

  init() {
    this.list.addEventListener('dragstart', (event) => {
      const item = event.target.closest('[draggable="true"]');
      if (!item) return;
      this.dragging = item;
      item.classList.add('is-dragging');
    });

    this.list.addEventListener('dragend', () => {
      if (this.dragging) this.dragging.classList.remove('is-dragging');
      this.dragging = null;
      this.emitOrder();
    });

    this.list.addEventListener('dragover', (event) => {
      event.preventDefault();
      const afterElement = this.getAfterElement(event.clientY);
      if (!this.dragging) return;
      if (afterElement == null) this.list.appendChild(this.dragging);
      else this.list.insertBefore(this.dragging, afterElement);
    });
  }

  getAfterElement(y) {
    const items = [...this.list.querySelectorAll('[draggable="true"]:not(.is-dragging)')];
    return items.reduce((closest, child) => {
      const box = child.getBoundingClientRect();
      const offset = y - box.top - box.height / 2;
      if (offset < 0 && offset > closest.offset) return { offset, element: child };
      return closest;
    }, { offset: Number.NEGATIVE_INFINITY }).element;
  }

  emitOrder() {
    const order = [...this.list.children].map((item) => item.dataset.id);
    if (this.onChange) this.onChange(order);
  }
}

// script.js
import { DragSorter } from './funciones.js';
new DragSorter('sortableList', (order) => console.log('Nuevo orden:', order));
```

JavaScript normal sin modulos

Usa esta version si prefieres tener todo en un solo archivo como script-normal.js y cargarlo con `<script src="script-normal.js"></script>`.

script-normal.js

```

class DragSorter {
  constructor(listId, onChange) {
    this.list = document.getElementById(listId);
    this.onChange = onChange;
    this.dragging = null;
    this.init();
  }
  init() {
    this.list.addEventListener('dragstart', (event) => {
      const item = event.target.closest('[draggable="true"]');
      if (!item) return;
      this.dragging = item;
      item.classList.add('is-dragging');
    });
    this.list.addEventListener('dragend', () => {
      if (this.dragging) this.dragging.classList.remove('is-dragging');
      this.dragging = null;
      this.emitOrder();
    });
    this.list.addEventListener('dragover', (event) => {
      event.preventDefault();
      const afterElement = this.getAfterElement(event.clientY);
      if (!this.dragging) return;
      if (afterElement == null) this.list.appendChild(this.dragging);
      else this.list.insertBefore(this.dragging, afterElement);
    });
  }
  getAfterElement(y) {
    const items = [...this.list.querySelectorAll('[draggable="true"]:not(.is-dragging)')];
    return items.reduce((closest, child) => {
      const box = child.getBoundingClientRect();
      const offset = y - box.top - box.height / 2;
      return offset < 0 && offset > closest.offset ? { offset, element: child } : closest;
    }, { offset: Number.NEGATIVE_INFINITY }).element;
  }
  emitOrder() {
    const order = [...this.list.children].map((item) => item.dataset.id);
    if (this.onChange) this.onChange(order);
  }
}
new DragSorter('sortableList', (order) => console.log('Nuevo orden:', order));

```

Variables, clases, ids y atributos que debes cambiar

Elemento	Para que sirve	Que debes cambiar
sortableList	Id del contenedor que contiene los elementos arrastrables.	Debe coincidir con el ul o div real.
draggable="true"	Activa la capacidad de arrastrar un elemento.	Debe ir en cada item.
data-id	Identificador real de cada item.	Usa ids de base de datos o ids internos.
is-dragging	Clase visual aplicada durante el arrastre.	Puedes cambiar opacidad, borde o escala.
onChange(order)	Callback que recibe el nuevo orden.	Usalo para guardar en localStorage o enviar al backend.

Implementacion paso a paso

1. Agrega draggable="true" a cada item que quieras mover.
2. Agrega data-id para reconocer cada elemento despues de moverlo.
3. Copia el CSS del estado normal y del estado is-dragging.
4. Copia DragSorter y pasale el id del contenedor.
5. En el callback onChange guarda el nuevo orden si lo necesitas.
6. Prueba mover elementos hacia arriba y hacia abajo.

Errores comunes y como solucionarlos

- No se mueve nada: falta `event.preventDefault()` en `dragover`.
- Se pierde el orden al recargar: debes guardar el array `order` en `localStorage` o backend.
- En movil no funciona igual: `drag and drop` nativo en `touch` puede requerir libreria o eventos `pointer`.
- Todos los items tienen el mismo `data-id`: usa identificadores unicos.

Checklist de prueba antes de reutilizar

- El componente existe en HTML y el id coincide con el usado en JavaScript.
- No hay ids duplicados en la pagina.
- La clase visual de estado se agrega y se quita correctamente.
- No aparecen errores en la consola del navegador.
- La funcionalidad sigue funcionando en pantalla pequena.
- El componente no rompe otros elementos del proyecto.

Mejoras opcionales

- Conectar el componente con un `Toast` para avisar acciones exitosas o errores.
- Guardar preferencias o datos en `localStorage` cuando tenga sentido.
- Agregar validaciones antes de ejecutar acciones importantes.
- Mejorar accesibilidad con `aria-label`, `aria-live`, `focus-visible` y navegacion por teclado cuando aplique.

43. Conversor C/F/K con botones de unidad

Aspecto	Explicacion
Que hace	Herramienta donde el usuario elige la unidad con botones C, F o K y luego calcula las conversiones restantes. Sirve como patron para apps donde un boton controla la logica.
Donde se usa en industria	Calculadoras, conversores, filtros rapidos, configuradores, formularios con opciones de modo, dashboards con cambio de metrica y herramientas educativas.
Como se personaliza	Cambia los botones data-unit, formulas, textos del resultado, ids del input y contenedor. Puedes reemplazar temperatura por moneda, medidas, porcentajes o cualquier calculo.

Mapa rapido de reciclaje

- 1 Ubica el lugar exacto del HTML donde quieres insertar el componente.
- 2 Copia el HTML minimo y cambia textos visibles, ids y data-attributes.
- 3 Copia el CSS y ajusta colores, separaciones, radios, sombras y responsive si hace falta.
- 4 Escoge JS modular o JS normal, no ambos al mismo tiempo.
- 5 Inicializa el componente con el id correcto y prueba la interaccion principal.

HTML minimo que debes copiar

index.html

```
<section class="panel temperature-tool" id="temperatureTool">
  <h2>Conversor de temperatura</h2>

  <div class="unit-buttons" role="group" aria-label="Unidad inicial">
    <button class="unit-button is-active" type="button" data-unit="celsius">C</button>
    <button class="unit-button" type="button" data-unit="fahrenheit">F</button>
    <button class="unit-button" type="button" data-unit="kelvin">K</button>
  </div>

  <input id="temperatureInput" type="number" placeholder="Ingrese temperatura">
  <button class="btn" id="temperatureCalculate" type="button">Calcular</button>
  <p class="message" id="temperatureMessage"></p>
  <div class="result-box" id="temperatureResult"></div>
</section>
```

CSS del componente

styles.css

```

.temperature-tool {
  display: grid;
  gap: 16px;
}
.unit-buttons {
  display: grid;
  grid-template-columns: repeat(3, 1fr);
  gap: 10px;
}
.unit-button {
  min-height: 46px;
  border: 1px solid rgba(255,255,255,.25);
  border-radius: 16px;
  color: #fff;
  background: rgba(255,255,255,.12);
  cursor: pointer;
  font-weight: 800;
  transition: transform .2s ease, background .2s ease;
}
.unit-button:hover,
.unit-button.is-active {
  transform: translateY(-2px);
  background: linear-gradient(135deg, #2563eb, #7c3aed);
}
.result-box {
  padding: 16px;
  border-radius: 18px;
  background: rgba(255,255,255,.12);
  border: 1px solid rgba(255,255,255,.18);
  line-height: 1.7;
}

```

JavaScript modular con export/import

Usa esta version si tu HTML carga `<script type="module" src="script.js"></script>`. La clase o funcion vive en `funciones.js` y la instancia se hace en `script.js`.

funciones.js + script.js

```

export class TemperatureTool {
  constructor(config) {
    this.buttons = document.querySelectorAll(config.buttonSelector);
    this.input = document.getElementById(config.inputId);
    this.calculateButton = document.getElementById(config.calculateId);
    this.message = document.getElementById(config.messageId);
    this.result = document.getElementById(config.resultId);
    this.unit = 'celsius';
    this.init();
  }

  init() {
    this.buttons.forEach((button) => {
      button.addEventListener('click', () => this.selectUnit(button));
    });
    this.calculateButton.addEventListener('click', () => this.calculate());
  }

  selectUnit(button) {
    this.unit = button.dataset.unit;
    this.buttons.forEach((item) => item.classList.remove('is-active'));
    button.classList.add('is-active');
  }

  calculate() {
    const value = Number(this.input.value.trim());
    if (!Number.isFinite(value)) {
      this.message.textContent = 'Ingresa un numero valido.';
      this.result.innerHTML = '';
      return;
    }
    this.message.textContent = 'Calculo realizado.';

    if (this.unit === 'celsius') {
      this.result.innerHTML = `Fahrenheit: ${((value * 9 / 5) + 32).toFixed(2)} F<br>Kelvin: ${((
value + 273.15).toFixed(2))} K`;
    }
    if (this.unit === 'fahrenheit') {
      const c = (value - 32) * 5 / 9;
      this.result.innerHTML = `Celsius: ${c.toFixed(2)} C<br>Kelvin: ${((c + 273.15).toFixed(2))} K`;
    }
    if (this.unit === 'kelvin') {
      const c = value - 273.15;
      this.result.innerHTML = `Celsius: ${c.toFixed(2)} C<br>Fahrenheit: ${(((c * 9 / 5) + 32).
toFixed(2))} F`;
    }
  }
}

// script.js
import { TemperatureTool } from './funciones.js';
new TemperatureTool({
  buttonSelector: '.unit-button', inputId: 'temperatureInput', calculateId: 'temperatureCalculate',
  messageId: 'temperatureMessage', resultId: 'temperatureResult'
});

```

JavaScript normal sin modulos

Usa esta version si prefieres tener todo en un solo archivo como script-normal.js y cargarlo con `<script src="script-normal.js"></script>`.

script-normal.js

```

class TemperatureTool {
  constructor(config) {
    this.buttons = document.querySelectorAll(config.buttonSelector);
    this.input = document.getElementById(config.inputId);
    this.calculateButton = document.getElementById(config.calculateId);
    this.message = document.getElementById(config.messageId);
    this.result = document.getElementById(config.resultId);
    this.unit = 'celsius';
    this.init();
  }
  init() {
    this.buttons.forEach((button) => button.addEventListener('click', () => this.selectUnit(
button)));
    this.calculateButton.addEventListener('click', () => this.calculate());
  }
  selectUnit(button) {
    this.unit = button.dataset.unit;
    this.buttons.forEach((item) => item.classList.remove('is-active'));
    button.classList.add('is-active');
  }
  calculate() {
    const value = Number(this.input.value.trim());
    if (!Number.isFinite(value)) {
      this.message.textContent = 'Ingresa un numero valido.';
      this.result.innerHTML = '';
      return;
    }
    this.message.textContent = 'Calculo realizado.';
    if (this.unit === 'celsius') this.result.innerHTML = `Fahrenheit: ${((value * 9 / 5) + 32).
toFixed(2)} F<br>Kelvin: ${((value + 273.15).toFixed(2))} K`;
    if (this.unit === 'fahrenheit') {
      const c = (value - 32) * 5 / 9;
      this.result.innerHTML = `Celsius: ${c.toFixed(2)} C<br>Kelvin: ${((c + 273.15).toFixed(2))} K`;
    }
    if (this.unit === 'kelvin') {
      const c = value - 273.15;
      this.result.innerHTML = `Celsius: ${c.toFixed(2)} C<br>Fahrenheit: ${(((c * 9 / 5) + 32).
toFixed(2))} F`;
    }
  }
}
new TemperatureTool({ buttonSelector: '.unit-button', inputId: 'temperatureInput', calculateId:
'temperatureCalculate', messageId: 'temperatureMessage', resultId: 'temperatureResult' });

```

Variables, clases, ids y atributos que debes cambiar

Elemento	Para que sirve	Que debes cambiar
data-unit	Define la unidad seleccionada por cada boton.	Valores esperados: celsius, fahrenheit, kelvin.
unit-button	Clase de los botones de unidad.	Usala para estilos y seleccion con querySelectorAll.
temperatureInput	Campo numerico principal.	Cambia si tu calculadora usa otro input.
temperatureResult	Caja donde se imprime el resultado.	Puedes usar innerHTML para multiples lineas.
this.unit	Estado actual de la unidad seleccionada.	Puedes iniciar en fahrenheit o kelvin si quieres.

Implementacion paso a paso

- 1 Copia el HTML de los tres botones y el input.
- 2 Asegurate de que cada boton tenga data-unit diferente.
- 3 Copia el CSS de unit-buttons, unit-button y result-box.
- 4 Copia la clase TemperatureTool y cambiale los ids en la instancia.
- 5 Prueba cada unidad con un valor conocido.
- 6 Si lo reciclas para otro calculo, reemplaza las formulas dentro de calculate().

Errores comunes y como solucionarlos

- Siempre calcula Celsius: revisa que data-unit este bien escrito.
- El boton no se activa visualmente: revisa la clase is-active.
- El resultado dice NaN: valida el input antes de calcular.
- innerHTML no aparece: revisa que resultId coincida con el HTML.

Checklist de prueba antes de reutilizar

- El componente existe en HTML y el id coincide con el usado en JavaScript.
- No hay ids duplicados en la pagina.
- La clase visual de estado se agrega y se quita correctamente.
- No aparecen errores en la consola del navegador.
- La funcionalidad sigue funcionando en pantalla pequena.
- El componente no rompe otros elementos del proyecto.

Mejoras opcionales

- Conectar el componente con un Toast para avisar acciones exitosas o errores.
- Guardar preferencias o datos en localStorage cuando tenga sentido.
- Agregar validaciones antes de ejecutar acciones importantes.
- Mejorar accesibilidad con aria-label, aria-live, focus-visible y navegacion por teclado cuando aplique.

44. Cambio de fondo segun boton seleccionado

Aspecto	Explicacion
Que hace	Cambia el estado visual global de la pagina agregando clases al body. En el ejemplo, C coloca fondo rojo, F azul y K verde.
Donde se usa en industria	Temas por categoria, pasos de un wizard, modo de visualizacion, cambio de marca, estados de calculadora, filtros destacados y pantallas interactivas.
Como se personaliza	Cambia las clases body.unit-celsius, body.unit-fahrenheit y body.unit-kelvin. Puedes usar cualquier nombre: mode-admin, theme-gold, state-success, etc.

Mapa rapido de reciclaje

- 1 Ubica el lugar exacto del HTML donde quieres insertar el componente.
- 2 Copia el HTML minimo y cambia textos visibles, ids y data-attributes.
- 3 Copia el CSS y ajusta colores, separaciones, radios, sombras y responsive si hace falta.
- 4 Escoge JS modular o JS normal, no ambos al mismo tiempo.
- 5 Inicializa el componente con el id correcto y prueba la interaccion principal.

HTML minimo que debes copiar

index.html

```
<div class="unit-buttons" role="group" aria-label="Cambiar fondo">
  <button class="unit-button" type="button" data-bg-class="unit-celsius">Fondo C</button>
  <button class="unit-button" type="button" data-bg-class="unit-fahrenheit">Fondo F</button>
  <button class="unit-button" type="button" data-bg-class="unit-kelvin">Fondo K</button>
</div>
```

CSS del componente

styles.css

```
body.unit-celsius {
  background: radial-gradient(circle at top left, #ef4444, transparent 35%),
    linear-gradient(135deg, #450a0a, #111827);
}
body.unit-fahrenheit {
  background: radial-gradient(circle at top left, #3b82f6, transparent 35%),
    linear-gradient(135deg, #082f49, #111827);
}
body.unit-kelvin {
  background: radial-gradient(circle at top left, #22c55e, transparent 35%),
    linear-gradient(135deg, #052e16, #111827);
}
body.unit-celsius .unit-button[data-bg-class="unit-celsius"],
body.unit-fahrenheit .unit-button[data-bg-class="unit-fahrenheit"],
body.unit-kelvin .unit-button[data-bg-class="unit-kelvin"] {
  background: rgba(255,255,255,.28);
  border-color: rgba(255,255,255,.75);
}
```

JavaScript modular con export/import

Usa esta version si tu HTML carga `<script type="module" src="script.js"></script>`. La clase o funcion vive en funciones.js y la instancia se hace en script.js.

funciones.js + script.js

```
export class BodyClassSwitcher {
  constructor(buttonSelector, allowedClasses) {
    this.buttons = document.querySelectorAll(buttonSelector);
    this.allowedClasses = allowedClasses;
    this.init();
  }

  init() {
    this.buttons.forEach((button) => {
      button.addEventListener('click', () => {
        this.setClass(button.dataset.bgClass);
      });
    });
  }

  setClass(className) {
    document.body.classList.remove(...this.allowedClasses);
    if (this.allowedClasses.includes(className)) {
      document.body.classList.add(className);
    }
  }
}

// script.js
import { BodyClassSwitcher } from './funciones.js';
new BodyClassSwitcher('[data-bg-class]', ['unit-celsius', 'unit-fahrenheit', 'unit-kelvin']);
```

JavaScript normal sin modulos

Usa esta version si prefieres tener todo en un solo archivo como script-normal.js y cargarlo con `<script src="script-normal.js"></script>`.

script-normal.js

```
class BodyClassSwitcher {
  constructor(buttonSelector, allowedClasses) {
    this.buttons = document.querySelectorAll(buttonSelector);
    this.allowedClasses = allowedClasses;
    this.init();
  }

  init() {
    this.buttons.forEach((button) => {
      button.addEventListener('click', () => this.setClass(button.dataset.bgClass));
    });
  }

  setClass(className) {
    document.body.classList.remove(...this.allowedClasses);
    if (this.allowedClasses.includes(className)) document.body.classList.add(className);
  }
}

new BodyClassSwitcher('[data-bg-class]', ['unit-celsius', 'unit-fahrenheit', 'unit-kelvin']);
```

Variables, clases, ids y atributos que debes cambiar

Elemento	Para que sirve	Que debes cambiar
data-bg-class	Clase que el boton quiere poner en body.	Debe existir en el CSS.
allowedClasses	Lista de clases permitidas.	Agrega todas las clases que se deben limpiar antes de cambiar.
document.body.classList	Controla clases globales del body.	No lo cambies salvo que quieras afectar otro contenedor.
body.unit-*	Reglas CSS que cambian fondo.	Cambia degradados, colores e imagenes.

Implementacion paso a paso

- 1 Define que clases visuales quieres usar en body.
- 2 Crea un boton por cada clase usando data-bg-class.

- 3 Copia las reglas CSS `body.nombreClase`.
- 4 Inicializa `BodyClassSwitcher` con el selector y array de clases permitidas.
- 5 Haz clic en cada boton y revisa que el body solo tenga una clase visual activa.

Errores comunes y como solucionarlos

- Se mezclan fondos: olvidaste quitar clases anteriores con `classList.remove(...allowedClasses)`.
- No cambia el fondo: el valor `data-bg-class` no coincide con la clase CSS.
- El cambio afecta toda la pagina: si quieres solo una seccion, aplica la clase a un contenedor en vez de `body`.
- El texto pierde contraste: ajusta colores de texto para cada fondo.

Checklist de prueba antes de reutilizar

- El componente existe en HTML y el id coincide con el usado en JavaScript.
- No hay ids duplicados en la pagina.
- La clase visual de estado se agrega y se quita correctamente.
- No aparecen errores en la consola del navegador.
- La funcionalidad sigue funcionando en pantalla pequena.
- El componente no rompe otros elementos del proyecto.

Mejoras opcionales

- Conectar el componente con un Toast para avisar acciones exitosas o errores.
- Guardar preferencias o datos en `localStorage` cuando tenga sentido.
- Agregar validaciones antes de ejecutar acciones importantes.
- Mejorar accesibilidad con `aria-label`, `aria-live`, `focus-visible` y navegacion por teclado cuando aplique.

45. Imprimir pagina o seccion

Aspecto	Explicacion
Que hace	Abre el dialogo de impresion del navegador. Es util para facturas, reportes, comprobantes, ordenes, hojas de vida, certificados o resúmenes.
Donde se usa en industria	Paneles administrativos, recibos, reportes internos, tickets, comprobantes de pedido, documentos de soporte y paginas de resumen.
Como se personaliza	Puedes imprimir toda la pagina con <code>window.print()</code> o preparar CSS @media print para ocultar botones, navbar y elementos que no deben aparecer.

Mapa rapido de reciclaje

- 1 Ubica el lugar exacto del HTML donde quieres insertar el componente.
- 2 Copia el HTML minimo y cambia textos visibles, ids y data-attributes.
- 3 Copia el CSS y ajusta colores, separaciones, radios, sombras y responsive si hace falta.
- 4 Escoge JS modular o JS normal, no ambos al mismo tiempo.
- 5 Inicializa el componente con el id correcto y prueba la interaccion principal.

HTML minimo que debes copiar

index.html

```
<section class="print-area" id="invoiceArea">
  <h2>Resumen imprimible</h2>
  <p>Cliente: Carlos Madero</p>
  <p>Total: $120.000</p>
</section>

<button class="btn no-print" id="printPage" type="button">Imprimir</button>
```

CSS del componente

styles.css

```
.print-area {
  padding: 24px;
  border-radius: 18px;
  background: rgba(255,255,255,.12);
  border: 1px solid rgba(255,255,255,.18);
}
@media print {
  body {
    background: #ffffff !important;
    color: #111827 !important;
  }
  .no-print,
  .navbar,
  .sidebar,
  .toast-container,
  .modal {
    display: none !important;
  }
  .print-area {
    background: #ffffff !important;
    color: #111827 !important;
    border: 1px solid #d1d5db !important;
    box-shadow: none !important;
  }
}
```

JavaScript modular con export/import

Usa esta version si tu HTML carga `<script type="module" src="script.js"></script>`. La clase o funcion vive en funciones.js y la instancia se hace en script.js.

funciones.js + script.js

```
export class PrintButton {
  constructor(buttonId, beforePrint = null) {
    this.button = document.getElementById(buttonId);
    this.beforePrint = beforePrint;
    this.init();
  }

  init() {
    this.button.addEventListener('click', () => {
      if (this.beforePrint) this.beforePrint();
      window.print();
    });
  }
}

// script.js
import { PrintButton } from './funciones.js';
new PrintButton('printPage', () => console.log('Preparando impresion...'));
```

JavaScript normal sin modulos

Usa esta version si prefieres tener todo en un solo archivo como script-normal.js y cargarlo con `<script src="script-normal.js"></script>`.

script-normal.js

```
class PrintButton {
  constructor(buttonId, beforePrint) {
    this.button = document.getElementById(buttonId);
    this.beforePrint = beforePrint;
    this.init();
  }

  init() {
    this.button.addEventListener('click', () => {
      if (this.beforePrint) this.beforePrint();
      window.print();
    });
  }
}

new PrintButton('printPage', () => console.log('Preparando impresion...'));
```

Variables, clases, ids y atributos que debes cambiar

Elemento	Para que sirve	Que debes cambiar
printPage	Boton que dispara la impresion.	Cambia el id si tienes varios botones.
no-print	Clase para ocultar elementos al imprimir.	Agregala a botones, menus o elementos decorativos.
@media print	CSS especial para impresion.	Ajusta colores, margenes y elementos visibles.
beforePrint	Callback antes de imprimir.	Usalo para calcular totales o cerrar modales.

Implementacion paso a paso

- 1 Agrega el boton de imprimir con id unico.
- 2 Agrega clase no-print a todo lo que no debe salir en papel.
- 3 Copia @media print en tu CSS.
- 4 Inicializa PrintButton en tu JS.
- 5 Prueba vista previa de impresion y ajusta margenes o colores.

Errores comunes y como solucionarlos

- Se imprime el menu: agrega no-print o regla display none en @media print.
- Los colores salen oscuros: redefine body y print-area en @media print.
- El boton no responde: revisa que el id printPage coincida.
- Quieres imprimir solo una seccion: debes usar CSS print para ocultar todo excepto esa seccion.

Checklist de prueba antes de reutilizar

- El componente existe en HTML y el id coincide con el usado en JavaScript.
- No hay ids duplicados en la pagina.
- La clase visual de estado se agrega y se quita correctamente.
- No aparecen errores en la consola del navegador.
- La funcionalidad sigue funcionando en pantalla pequena.
- El componente no rompe otros elementos del proyecto.

Mejoras opcionales

- Conectar el componente con un Toast para avisar acciones exitosas o errores.
- Guardar preferencias o datos en localStorage cuando tenga sentido.
- Agregar validaciones antes de ejecutar acciones importantes.
- Mejorar accesibilidad con aria-label, aria-live, focus-visible y navegacion por teclado cuando aplique.

46. Reset visual de estados y clases

Aspecto	Explicacion
Que hace	Limpia clases temporales de la interfaz para regresar a un estado base. Es util cuando muchos botones cambian fondos, temas, filtros o estados visuales.
Donde se usa en industria	Dashboards con filtros, calculadoras, formularios multi-paso, juegos sencillos, prototipos de UI, reset de demos y limpieza de estados visuales.
Como se personaliza	Cambia el array de clases a limpiar, el boton de reset y si tambien quieres limpiar inputs, resultados o localStorage.

Mapa rapido de reciclaje

- 1 Ubica el lugar exacto del HTML donde quieres insertar el componente.
- 2 Copia el HTML minimo y cambia textos visibles, ids y data-attributes.
- 3 Copia el CSS y ajusta colores, separaciones, radios, sombras y responsive si hace falta.
- 4 Escoge JS modular o JS normal, no ambos al mismo tiempo.
- 5 Inicializa el componente con el id correcto y prueba la interaccion principal.

HTML minimo que debes copiar

index.html

```
<button class="btn" id="resetVisual" type="button">Reset visual</button>  
<p id="resetMessage" class="message"></p>
```

CSS del componente

styles.css

```
.message {  
  min-height: 20px;  
  color: rgba(255,255,255,.8);  
}  
body.unit-celsius,  
body.unit-fahrenheit,  
body.unit-kelvin {  
  transition: background .35s ease, color .35s ease;  
}
```

JavaScript modular con export/import

Usa esta version si tu HTML carga `<script type="module" src="script.js"></script>`. La clase o funcion vive en funciones.js y la instancia se hace en script.js.

funciones.js + script.js

```
export class VisualReset {
  constructor(config) {
    this.button = document.getElementById(config.buttonId);
    this.message = document.getElementById(config.messageId);
    this.target = config.target || document.body;
    this.classesToRemove = config.classesToRemove;
    this.extraReset = config.extraReset || null;
    this.init();
  }

  init() {
    this.button.addEventListener('click', () => this.reset());
  }

  reset() {
    this.target.classList.remove(...this.classesToRemove);
    if (this.extraReset) this.extraReset();
    if (this.message) this.message.textContent = 'Vista reiniciada.';
  }
}

// script.js
import { VisualReset } from './funciones.js';
new VisualReset({
  buttonId: 'resetVisual',
  messageId: 'resetMessage',
  classesToRemove: ['unit-celsius', 'unit-fahrenheit', 'unit-kelvin']
});
```

JavaScript normal sin modulos

Usa esta version si prefieres tener todo en un solo archivo como script-normal.js y cargarlo con `<script src="script-normal.js"></script>`.

script-normal.js

```
class VisualReset {
  constructor(config) {
    this.button = document.getElementById(config.buttonId);
    this.message = document.getElementById(config.messageId);
    this.target = config.target || document.body;
    this.classesToRemove = config.classesToRemove;
    this.extraReset = config.extraReset || null;
    this.init();
  }
  init() { this.button.addEventListener('click', () => this.reset()); }
  reset() {
    this.target.classList.remove(...this.classesToRemove);
    if (this.extraReset) this.extraReset();
    if (this.message) this.message.textContent = 'Vista reiniciada.';
  }
}
new VisualReset({ buttonId: 'resetVisual', messageId: 'resetMessage', classesToRemove: ['unit-celsius', 'unit-fahrenheit', 'unit-kelvin'] });
```

Variables, clases, ids y atributos que debes cambiar

Elemento	Para que sirve	Que debes cambiar
buttonId	Boton que ejecuta el reset.	Debe coincidir con el id HTML.
classesToRemove	Array con clases que se limpian.	Agrega aqui todos los estados visuales temporales.
target	Elemento al que se le quitan clases.	Por defecto es document.body.
extraReset	Funcion opcional para limpiar inputs o resultados.	Usala si quieres resetear mas cosas.

Implementacion paso a paso

- 1 Identifica las clases visuales temporales del proyecto.
- 2 Crea el boton de reset.

- 3 Copia la clase VisualReset.
- 4 Pasa el array de clases en classesToRemove.
- 5 Si necesitas limpiar campos, agrega extraReset.
- 6 Prueba activar varios estados y luego reiniciarlos.

Errores comunes y como solucionarlos

- No limpia nada: la clase en classesToRemove no coincide con la clase real.
- Limpia demasiado: no metas clases base del layout en el array.
- El mensaje no aparece: messageId no coincide o el elemento no existe.
- El reset visual no borra datos: solo borra clases; usa extraReset para datos.

Checklist de prueba antes de reutilizar

- El componente existe en HTML y el id coincide con el usado en JavaScript.
- No hay ids duplicados en la pagina.
- La clase visual de estado se agrega y se quita correctamente.
- No aparecen errores en la consola del navegador.
- La funcionalidad sigue funcionando en pantalla pequena.
- El componente no rompe otros elementos del proyecto.

Mejoras opcionales

- Conectar el componente con un Toast para avisar acciones exitosas o errores.
- Guardar preferencias o datos en localStorage cuando tenga sentido.
- Agregar validaciones antes de ejecutar acciones importantes.
- Mejorar accesibilidad con aria-label, aria-live, focus-visible y navegacion por teclado cuando aplique.

47. Helpers DOM y storage

Aspecto	Explicacion
Que hace	Pequeñas funciones reutilizables para escribir menos código repetido al seleccionar elementos, guardar JSON, leer JSON, mostrar mensajes o formatear datos.
Donde se usa en industria	Cualquier proyecto frontend: landing pages, formularios, dashboards, e-commerce, proyectos académicos, prototipos y aplicaciones SPA simples.
Como se personaliza	Puedes agregar helpers nuevos según tu estilo: validarEmail, formatCurrency, createElement, showError, clearHTML, debounce, throttle, etc.

Mapa rapido de reciclaje

- 1 Ubica el lugar exacto del HTML donde quieres insertar el componente.
- 2 Copia el HTML mínimo y cambia textos visibles, ids y data-attributes.
- 3 Copia el CSS y ajusta colores, separaciones, radios, sombras y responsive si hace falta.
- 4 Escoge JS modular o JS normal, no ambos al mismo tiempo.
- 5 Inicializa el componente con el id correcto y prueba la interacción principal.

HTML mínimo que debes copiar

index.html

```
<p id="helperMessage"></p>
<button class="btn" id="helperDemo" type="button">Probar helper</button>
```

CSS del componente

styles.css

```
.helper-ok {
  color: #22c55e;
  font-weight: 700;
}
.helper-error {
  color: #ef4444;
  font-weight: 700;
}
```

JavaScript modular con export/import

Usa esta versión si tu HTML carga `<script type="module" src="script.js"></script>`. La clase o función vive en `funciones.js` y la instancia se hace en `script.js`.

funciones.js + script.js

```

export const $ = (selector, parent = document) => parent.querySelector(selector);
export const $$ = (selector, parent = document) => [...parent.querySelectorAll(selector)];

export function saveJSON(key, value) {
  localStorage.setItem(key, JSON.stringify(value));
}

export function readJSON(key, fallback = null) {
  try {
    const value = localStorage.getItem(key);
    return value ? JSON.parse(value) : fallback;
  } catch (error) {
    console.error('Error leyendo JSON:', error);
    return fallback;
  }
}

export function showMessage(element, text, type = 'ok') {
  element.textContent = text;
  element.classList.remove('helper-ok', 'helper-error');
  element.classList.add(type === 'ok' ? 'helper-ok' : 'helper-error');
}

export function formatMoney(value, locale = 'es-CO', currency = 'COP') {
  return new Intl.NumberFormat(locale, { style: 'currency', currency }).format(value);
}

// script.js
import { $, saveJSON, readJSON, showMessage, formatMoney } from './funciones.js';
$('#helperDemo').addEventListener('click', () => {
  saveJSON('demo-price', { price: 12000 });
  const data = readJSON('demo-price', { price: 0 });
  showMessage($('#helperMessage'), `Precio: ${formatMoney(data.price)}`, 'ok');
});

```

JavaScript normal sin modulos

Usa esta version si prefieres tener todo en un solo archivo como script-normal.js y cargarlo con `<script src="script-normal.js"></script>`.

script-normal.js

```

const $ = (selector, parent = document) => parent.querySelector(selector);
const $$ = (selector, parent = document) => [...parent.querySelectorAll(selector)];
function saveJSON(key, value) {
  localStorage.setItem(key, JSON.stringify(value));
}
function readJSON(key, fallback = null) {
  try {
    const value = localStorage.getItem(key);
    return value ? JSON.parse(value) : fallback;
  } catch (error) {
    console.error('Error leyendo JSON:', error);
    return fallback;
  }
}
function showMessage(element, text, type = 'ok') {
  element.textContent = text;
  element.classList.remove('helper-ok', 'helper-error');
  element.classList.add(type === 'ok' ? 'helper-ok' : 'helper-error');
}
function formatMoney(value, locale = 'es-CO', currency = 'COP') {
  return new Intl.NumberFormat(locale, { style: 'currency', currency }).format(value);
}
$('#helperDemo').addEventListener('click', () => {
  saveJSON('demo-price', { price: 12000 });
  const data = readJSON('demo-price', { price: 0 });
  showMessage($('#helperMessage'), `Precio: ${formatMoney(data.price)}`, 'ok');
});

```

Variables, clases, ids y atributos que debes cambiar

Elemento	Para que sirve	Que debes cambiar
\$	Selecciona el primer elemento que coincide.	Usalo como atajo de querySelector.

Elemento	Para que sirve	Que debes cambiar
\$\$	Selecciona varios elementos y los vuelve array.	Util para forEach, map, filter.
saveJSON / readJSON	Guarda y lee objetos en localStorage.	Cambia la key segun el proyecto.
showMessage	Muestra texto y aplica clase de estado.	Cambia clases helper-ok/helper-error.
formatMoney	Formatea dinero segun pais y moneda.	Cambia locale y currency.

Implementacion paso a paso

- 1 Copia los helpers al inicio de funciones.js o script-normal.js.
- 2 Exportalos si trabajas modular.
- 3 Usalos en tus componentes para evitar repetir codigo.
- 4 Agrega nuevos helpers cuando una accion se repita varias veces.
- 5 No crees helpers demasiado especificos si solo los usaras una vez.

Errores comunes y como solucionarlos

- \$ no existe: en modular debes importarlo antes de usarlo.
- readJSON falla: el contenido guardado no es JSON valido.
- localStorage no guarda objetos directamente: usa JSON.stringify como en saveJSON.
- formatMoney muestra otra moneda: cambia locale y currency.

Checklist de prueba antes de reutilizar

- El componente existe en HTML y el id coincide con el usado en JavaScript.
- No hay ids duplicados en la pagina.
- La clase visual de estado se agrega y se quita correctamente.
- No aparecen errores en la consola del navegador.
- La funcionalidad sigue funcionando en pantalla pequena.
- El componente no rompe otros elementos del proyecto.

Mejoras opcionales

- Conectar el componente con un Toast para avisar acciones exitosas o errores.
- Guardar preferencias o datos en localStorage cuando tenga sentido.
- Agregar validaciones antes de ejecutar acciones importantes.
- Mejorar accesibilidad con aria-label, aria-live, focus-visible y navegacion por teclado cuando aplique.

48. Version modular con export/import

Aspecto	Explicacion
Que hace	Organiza el codigo en archivos separados. funciones.js contiene clases o utilidades exportadas y script.js importa lo necesario para iniciar la pagina.
Donde se usa en industria	Frontend moderno, proyectos escalables, equipos de desarrollo, componentes reutilizables, separacion de responsabilidades y mantenimiento limpio.
Como se personaliza	Cambia los nombres de clases, rutas de importacion y funciones exportadas. Recuerda cargar el script principal con type="module".

Mapa rapido de reciclaje

- 1 Ubica el lugar exacto del HTML donde quieres insertar el componente.
- 2 Copia el HTML minimo y cambia textos visibles, ids y data-attributes.
- 3 Copia el CSS y ajusta colores, separaciones, radios, sombras y responsive si hace falta.
- 4 Escoge JS modular o JS normal, no ambos al mismo tiempo.
- 5 Inicializa el componente con el id correcto y prueba la interaccion principal.

HTML minimo que debes copiar

index.html

```
<!-- index.html -->
<script type="module" src="script.js"></script>
```

CSS del componente

styles.css

```
/* CSS no cambia por usar modulos.
   La modularidad es una forma de organizar JavaScript. */
.app {
  min-height: 100vh;
}
```

JavaScript modular con export/import

Usa esta version si tu HTML carga <script type="module" src="script.js"></script>. La clase o funcion vive en funciones.js y la instancia se hace en script.js.

funciones.js + script.js

```
// funciones.js
export class SaludoApp {
  constructor(messageId) {
    this.message = document.getElementById(messageId);
  }

  show(name) {
    this.message.textContent = `Hola, ${name}`;
  }
}

export function cleanText(value) {
  return value.trim();
}

// script.js
import { SaludoApp, cleanText } from './funciones.js';

const app = new SaludoApp('mensaje');
const nombre = cleanText(' Carlos ');
app.show(nombre);
```

JavaScript normal sin modulos

Usa esta version si prefieres tener todo en un solo archivo como script-normal.js y cargarlo con `<script src="script-normal.js"></script>`.

script-normal.js

```
// En version normal no usas export ni import.
// Todo queda disponible en el mismo archivo script-normal.js.
class SaludoApp {
  constructor(messageId) {
    this.message = document.getElementById(messageId);
  }
  show(name) {
    this.message.textContent = `Hola, ${name}`;
  }
}
function cleanText(value) {
  return value.trim();
}
const app = new SaludoApp('mensaje');
const nombre = cleanText(' Carlos ');
app.show(nombre);
```

Variables, clases, ids y atributos que debes cambiar

Elemento	Para que sirve	Que debes cambiar
type="module"	Activa soporte de import/export en el navegador.	Debe ir en el script principal.
export	Permite sacar una clase o funcion de un archivo.	Ponlo delante de class, function o const.
import	Trae codigo desde otro archivo.	La ruta debe empezar con ./ si es archivo local.
./funciones.js	Ruta del archivo que exporta.	Cambia si tu archivo se llama distinto.
script.js	Archivo que arranca la aplicacion.	Debe importar y crear instancias.

Implementacion paso a paso

- 1 Crea funciones.js para clases y funciones reutilizables.
- 2 Exporta cada clase o funcion que necesites usar afuera.
- 3 Crea script.js como archivo de arranque.
- 4 Importa lo necesario desde funciones.js.

- 5 En HTML carga script.js con type="module".
- 6 Ejecuta con Live Server; no abras el HTML como archivo local si hay problemas de modulo.

Errores comunes y como solucionarlos

- Cannot use import statement outside a module: falta type="module".
- Failed to resolve module specifier: la ruta del import esta mal.
- No se encuentra una clase: falta exportarla o importarla con el mismo nombre.
- Se ejecuta antes del HTML: coloca el script al final del body o usa DOMContentLoaded.

Checklist de prueba antes de reutilizar

- El componente existe en HTML y el id coincide con el usado en JavaScript.
- No hay ids duplicados en la pagina.
- La clase visual de estado se agrega y se quita correctamente.
- No aparecen errores en la consola del navegador.
- La funcionalidad sigue funcionando en pantalla pequena.
- El componente no rompe otros elementos del proyecto.

Mejoras opcionales

- Conectar el componente con un Toast para avisar acciones exitosas o errores.
- Guardar preferencias o datos en localStorage cuando tenga sentido.
- Agregar validaciones antes de ejecutar acciones importantes.
- Mejorar accesibilidad con aria-label, aria-live, focus-visible y navegacion por teclado cuando aplique.

49. Version normal sin modulos

Aspecto	Explicacion
Que hace	Permite usar JavaScript en un solo archivo sin export/import. Es util para ejercicios simples, entregas basicas o proyectos donde no quieres configurar modulos.
Donde se usa en industria	Landing pages pequenas, ejercicios academicos, scripts de una sola pagina, prototipos rapidos, integraciones muy simples y demos.
Como se personaliza	Cambia el nombre del archivo script-normal.js y el orden de carga. Si separas varios scripts normales, el orden importa mucho.

Mapa rapido de reciclaje

- 1 Ubica el lugar exacto del HTML donde quieres insertar el componente.
- 2 Copia el HTML minimo y cambia textos visibles, ids y data-attributes.
- 3 Copia el CSS y ajusta colores, separaciones, radios, sombras y responsive si hace falta.
- 4 Escoge JS modular o JS normal, no ambos al mismo tiempo.
- 5 Inicializa el componente con el id correcto y prueba la interaccion principal.

HTML minimo que debes copiar

index.html

```
<!-- index.html -->
<script src="script-normal.js"></script>
```

CSS del componente

styles.css

```
/* CSS tampoco cambia por usar version normal. */
.page-ready {
  opacity: 1;
}
```

JavaScript modular con export/import

Usa esta version si tu HTML carga `<script type="module" src="script.js"></script>`. La clase o funcion vive en funciones.js y la instancia se hace en script.js.

funciones.js + script.js

```
// Equivalencia conceptual en modular:
// funciones.js
export class AppNormalEquivalente {
  start() {
    console.log('App iniciada en modo modular');
  }
}

// script.js
import { AppNormalEquivalente } from './funciones.js';
new AppNormalEquivalente().start();
```

JavaScript normal sin modulos

Usa esta version si prefieres tener todo en un solo archivo como script-normal.js y cargarlo con <script src="script-normal.js"></script>.

script-normal.js

```
// script-normal.js
class AppNormal {
  start() {
    console.log('App iniciada sin modulos');
  }
}

function initApp() {
  const app = new AppNormal();
  app.start();
}

// Opcion 1: si el script esta al final del body
initApp();

// Opcion 2: si el script esta en el head
// document.addEventListener('DOMContentLoaded', initApp);
```

Variables, clases, ids y atributos que debes cambiar

Elemento	Para que sirve	Que debes cambiar
script-normal.js	Archivo unico donde vive todo el codigo.	Puedes cambiar el nombre, pero actualiza el src.
DOMContentLoaded	Evento que espera a que cargue el HTML.	Usalo si el script esta en head.
orden de scripts	En JS normal, lo de arriba debe existir antes de usarse.	Carga helpers antes que componentes que los usan.
variables globales	Todo puede quedar expuesto en window si usas var.	Prefiere const, let y clases para evitar choques.

Implementacion paso a paso

- 1 Crea un archivo script-normal.js.
- 2 Pega helpers, clases y luego el codigo de inicializacion.
- 3 Carga el script al final del body.
- 4 No uses import ni export en este archivo.
- 5 Si tienes varios archivos normales, carga primero helpers, despues clases, despues script principal.

Errores comunes y como solucionarlos

- Unexpected token export: estas usando export en un script normal.
- Clase no definida: cargaste los scripts en mal orden.
- document.getElementById retorna null: el script se ejecuto antes de que exista el HTML.
- Choques de nombres: evita variables globales genericas como data, item, app si hay muchos scripts.

Checklist de prueba antes de reutilizar

- El componente existe en HTML y el id coincide con el usado en JavaScript.
- No hay ids duplicados en la pagina.
- La clase visual de estado se agrega y se quita correctamente.
- No aparecen errores en la consola del navegador.

- La funcionalidad sigue funcionando en pantalla pequena.
- El componente no rompe otros elementos del proyecto.

Mejoras opcionales

- Conectar el componente con un Toast para avisar acciones exitosas o errores.
- Guardar preferencias o datos en localStorage cuando tenga sentido.
- Agregar validaciones antes de ejecutar acciones importantes.
- Mejorar accesibilidad con aria-label, aria-live, focus-visible y navegacion por teclado cuando aplique.

50. Responsive design reutilizable

Aspecto	Explicacion
Que hace	Hace que la pagina se adapte a celular, tablet y escritorio. Es una de las partes mas importantes para que una web se vea profesional.
Donde se usa en industria	Todas las paginas web modernas: landings, portafolios, ecommerce, dashboards, formularios, blogs, sistemas internos y aplicaciones web.
Como se personaliza	Cambia breakpoints, grid, tamanos de fuente, padding, columnas y comportamiento de navbar/sidebar. Prueba siempre en menos de 520px.

Mapa rapido de reciclaje

- 1 Ubica el lugar exacto del HTML donde quieres insertar el componente.
- 2 Copia el HTML minimo y cambia textos visibles, ids y data-attributes.
- 3 Copia el CSS y ajusta colores, separaciones, radios, sombras y responsive si hace falta.
- 4 Escoge JS modular o JS normal, no ambos al mismo tiempo.
- 5 Inicializa el componente con el id correcto y prueba la interaccion principal.

HTML minimo que debes copiar

index.html

```
<main class="responsive-layout">
  <section class="hero-card">Contenido principal</section>
  <aside class="side-card">Contenido secundario</aside>
</main>
```

CSS del componente

styles.css

```

:root {
  --container: 1120px;
  --space: 24px;
  --radius: 22px;
}
.responsive-layout {
  width: min(var(--container), calc(100% - 32px));
  margin: 0 auto;
  display: grid;
  grid-template-columns: minmax(0, 2fr) minmax(280px, 1fr);
  gap: var(--space);
}
.hero-card,
.side-card {
  padding: 24px;
  border-radius: var(--radius);
  background: rgba(255,255,255,.12);
  border: 1px solid rgba(255,255,255,.18);
}
@media (max-width: 900px) {
  .responsive-layout {
    grid-template-columns: 1fr;
  }
}
@media (max-width: 520px) {
  :root {
    --space: 14px;
  }
  .responsive-layout {
    width: min(100% - 20px, var(--container));
  }
  .hero-card,
  .side-card {
    padding: 18px;
    border-radius: 16px;
  }
  h1 {
    font-size: clamp(1.7rem, 9vw, 2.4rem);
  }
}

```

JavaScript modular con export/import

Usa esta version si tu HTML carga `<script type="module" src="script.js"></script>`. La clase o funcion vive en funciones.js y la instancia se hace en script.js.

funciones.js + script.js

```

export class ResponsiveHelper {
  constructor(outputId) {
    this.output = document.getElementById(outputId);
    this.init();
  }

  init() {
    this.update();
    window.addEventListener('resize', () => this.update());
  }

  update() {
    const width = window.innerWidth;
    let label = 'desktop';
    if (width <= 520) label = 'mobile';
    else if (width <= 900) label = 'tablet';
    if (this.output) this.output.textContent = `Vista actual: ${label} (${width}px)`;
    document.body.dataset.viewport = label;
  }
}

// script.js
import { ResponsiveHelper } from './funciones.js';
new ResponsiveHelper('viewportLabel');

```

JavaScript normal sin modulos

Usa esta version si prefieres tener todo en un solo archivo como script-normal.js y cargarlo con `<script src="script-normal.js"></script>`.

script-normal.js

```
class ResponsiveHelper {
  constructor(outputId) {
    this.output = document.getElementById(outputId);
    this.init();
  }
  init() {
    this.update();
    window.addEventListener('resize', () => this.update());
  }
  update() {
    const width = window.innerWidth;
    let label = 'desktop';
    if (width <= 520) label = 'mobile';
    else if (width <= 900) label = 'tablet';
    if (this.output) this.output.textContent = `Vista actual: ${label} (${width}px)`;
    document.body.dataset.viewport = label;
  }
}
new ResponsiveHelper('viewportLabel');
```

Variables, clases, ids y atributos que debes cambiar

Elemento	Para que sirve	Que debes cambiar
--container	Ancho maximo del contenido.	Cambia segun diseno: 960px, 1120px, 1280px.
breakpoints	Puntos donde cambia el layout.	Ajusta 900px y 520px segun tu pagina.
grid-template-columns	Define columnas en desktop.	Cambia 2fr/1fr por 1fr/1fr o repeat.
clamp()	Controla tamanos fluidos.	Util para titulos responsive.
data-viewport	Etiqueta el estado actual desde JS.	Puedes usarla para depurar o cambiar estilos.

Implementacion paso a paso

- 1 Empieza disenando mobile o desktop, pero prueba ambos.
- 2 Usa contenedores con width: min(...) para evitar desbordes.
- 3 Convierte grids de varias columnas a una columna en tablet/mobile.
- 4 Reduce padding y radios en pantallas pequenas.
- 5 Prueba navbar, sidebar, formularios, modales y tablas en menos de 520px.
- 6 Usa DevTools para revisar distintos anchos.

Errores comunes y como solucionarlos

- Aparece scroll horizontal: revisa widths fijos, imagenes grandes o grid sin minmax(0, 1fr).
- El texto queda gigante en movil: usa clamp o media queries.
- Las tablas se rompen: agrega overflow-x:auto al contenedor.
- El sidebar tapa todo: define comportamiento mobile para menus fijos.

Checklist de prueba antes de reutilizar

- El componente existe en HTML y el id coincide con el usado en JavaScript.
- No hay ids duplicados en la pagina.
- La clase visual de estado se agrega y se quita correctamente.
- No aparecen errores en la consola del navegador.
- La funcionalidad sigue funcionando en pantalla pequena.

- El componente no rompe otros elementos del proyecto.

Mejoras opcionales

- Conectar el componente con un Toast para avisar acciones exitosas o errores.
- Guardar preferencias o datos en localStorage cuando tenga sentido.
- Agregar validaciones antes de ejecutar acciones importantes.
- Mejorar accesibilidad con aria-label, aria-live, focus-visible y navegacion por teclado cuando aplique.

Cierre: como usar las funcionalidades 41 a 50 en proyectos reales

Estas ultimas funcionalidades completan el banco porque cubren tareas muy comunes: guardar tareas, reordenar elementos, calcular valores, cambiar estados visuales, imprimir, reiniciar interfaces, usar helpers, decidir entre JS modular o normal y hacer responsive el proyecto. Si entiendes estas piezas, puedes armar muchas paginas profesionales sin empezar desde cero.

Patron recomendado para un proyecto reutilizable

Ejemplo de integracion modular

```
// 1. HTML define estructura e ids.
// 2. CSS define apariencia y estados visuales.
// 3. funciones.js guarda clases reutilizables.
// 4. script.js importa e inicializa.
// 5. localStorage guarda estado si el usuario debe conservar datos.
// 6. @media adapta el diseno a celular.

import { TodoList, DragSorter, TemperatureTool, BodyClassSwitcher, VisualReset } from './funciones.js';

new TodoList({ formId: 'todoForm', inputId: 'todoInput', listId: 'todoList', storageKey: 'project-todos'
});
new DragSorter('todoList', (order) => console.log(order));
new TemperatureTool({
  buttonSelector: '.unit-button',
  inputId: 'temperatureInput',
  calculateId: 'temperatureCalculate',
  messageId: 'temperatureMessage',
  resultId: 'temperatureResult'
});
new BodyClassSwitcher('[data-bg-class]', ['unit-celsius', 'unit-fahrenheit', 'unit-kelvin']);
new VisualReset({ buttonId: 'resetVisual', classesToRemove: ['unit-celsius', 'unit-fahrenheit', 'unit-kelvin'] });
```

Checklist final de integracion

- Cada id del HTML existe una sola vez.
- Cada clase usada por JavaScript existe en CSS si necesita estilo visual.
- Cada data-attribute tiene el mismo nombre que lee JavaScript.
- Las claves de localStorage son unicas por proyecto.
- No mezclas import/export con script normal en el mismo flujo.
- La pagina se prueba en desktop, tablet y celular.
- La consola del navegador no muestra errores.

Actualizacion funcional del bloque 41 a 57

Este anexo actualiza los codigos de copiar y pegar para que queden alineados con la pagina actual y funcionen como referencia implementable.

- Usa estos bloques como fuente de consulta y copiar/pegar para HTML, CSS y JavaScript.
- El contenido anterior del PDF base se conserva, pero este anexo manda para implementacion.

51. Confirmación visual

¿Que hace?

Agrega retroalimentación inmediata después de una acción: guardar, eliminar, copiar, iniciar sesión o terminar una validación. En este proyecto normalmente se resuelve con toasts, badges de estado o mensajes cortos cerca del componente.

¿Cuándo usarlo?

Usalo cuando quieras que el usuario sepa de inmediato si una acción salió bien, si faltó un dato o si algo cambió en la interfaz.

Paso a paso para reciclarlo en otro proyecto

1. Define que acciones deben mostrar confirmación: guardar, borrar, copiar o actualizar.
2. Elige el canal visual: toast flotante, badge temporal, texto de estado o combinación de ellos.
3. Mantén un mensaje corto y claro, por ejemplo: "Nota guardada." o "Código copiado."
4. Dispara la confirmación justo después de que la acción termine correctamente.
5. Si también hay error, usa un mensaje alternativo como "No se pudo guardar" o "Revisa los campos".
6. Prueba tanto el caso exitoso como el caso fallido para que la retroalimentación no engañe al usuario.

Checklist recomendado

- El mensaje aparece en el momento correcto.
- El usuario entiende que pasó sin releer la pantalla completa.
- El color y el contraste siguen siendo legibles en tema oscuro y claro.
- La confirmación desaparece sola o puede cerrarse fácilmente.

Con que combinarlo

Esta funcionalidad encaja muy bien con Toast, Login demo, Notas, Todo list, Copy clipboard y Reset demo.

Paso a paso detallado para construirlo desde cero

1. Identifica en tu proyecto el lugar exacto donde necesitas insertar el componente. Analiza si va en el header, en el main, en un sidebar o en un modal.
2. Crea el HTML mínimo con los ids y clases correctas. Si copias de este banco, asegúrate de que no haya ids duplicados en tu página.
3. Copia el CSS que define la apariencia del componente. Presta atención a las variables CSS que controlan colores, espacios, radios y sombras.
4. Si el componente usa animaciones o transiciones, verifica que las propiedades `transition` y `animation` estén definidas en CSS.
5. Copia la clase JavaScript modular o la función normal. Si usas módulos, asegúrate de que `script.js` tenga `type="module"` y que las rutas de import sean correctas.
6. Inicializa el componente pasando los ids correctos. Si el constructor espera un id

- y tu HTML tiene otro, cambia el argumento al crear la instancia.
7. Prueba el componente en el navegador. Abre la consola (F12) y verifica que no haya errores rojos.
 8. Si hay errores, revisa que los elementos existan en el DOM antes de que JavaScript intente usarlos. Coloca el script al final del body o usa DOMContentLoaded.
 9. Prueba el componente en diferentes resoluciones: escritorio, tablet y movil. Usa DevTools para simular distintos anchos de pantalla.
 10. Si todo funciona correctamente, guarda una copia de seguridad y documenta cualquier personalizacion que hayas hecho.

Errores comunes y como solucionarlos

El componente no aparece: revisa que el HTML tenga el id correcto y que el script se cargue despues de que el HTML exista. Si el script esta en el head, el elemento no existe todavia cuando JavaScript intenta seleccionarlo.

El estilo no se aplica: revisa que las clases CSS del componente esten en styles.css y que no haya errores de sintaxis que rompan el resto del archivo.

La interaccion no funciona: revisa que los event listeners se conecten a elementos que realmente existen. Si usas querySelector, confirma que el selector coincida con el HTML.

El componente se ve mal en movil: revisa las reglas @media. Asegurate de que los anchos fijos, imagenes grandes o grids sin minmax no causen scroll horizontal.

Conflictos con otros componentes: si tienes varios componentes con ids o clases similares, puede haber choques. Usa nombres descriptivos y unicos para cada componente.

Checklist de prueba antes de reutilizar

El componente existe en HTML y el id coincide con el usado en JavaScript.

No hay ids duplicados en la pagina.

La clase visual de estado se agrega y se quita correctamente.

No aparecen errores en la consola del navegador.

La funcionalidad sigue funcionando en pantalla pequena.

El componente no rompe otros elementos del proyecto.

El CSS no genera scroll horizontal en ninguna resolucion.

Los eventos responden al clic, al teclado o al scroll segun corresponda.

Que se puede reutilizar directamente

El patron de estructura HTML + CSS + JS de esta funcionalidad se puede adaptar para cualquier componente similar. La logica de seleccion de elementos, manejo de eventos y actualizacion del DOM es un patron universal que se repite en la mayoria de los componentes web.

La forma de organizar el codigo en clases con constructor, init y metodos especificos se puede copiar para crear nuevos componentes. Solo cambias los selectores, los eventos y la logica interna.

Las variables CSS personalizadas se pueden reutilizar en otros componentes para mantener consistencia de colores, espacios y radios.

Variables, clases, ids y atributos que debes cambiar

- ****Elemento**** (Para que sirve): Que debes cambiar
- ****id principal**** (identifica el contenedor del componente): Cambia si ya existe ese id en tu pagina
- ****clases CSS**** (definen la apariencia visual): Cambia si colisionan con otras clases
- ****data-***** (atributos personalizados para JS): Cambia los valores segun tus necesidades
- ****textos visibles**** (etiquetas, titulos, botones): Cambia a los textos de tu proyecto
- ****colores**** (variables CSS o valores hardcodeados): Cambia para que coincidan con tu marca

52. Debounce

¿Que hace?

Retrasa la ejecución de una función hasta que el usuario deja de escribir o de disparar eventos repetidos durante unos milisegundos. Sirve para búsquedas, filtros, autoguardado, validaciones en tiempo real y renderizados costosos.

¿Cuándo usarlo?

Usalo cuando un input dispara demasiadas actualizaciones seguidas y quieres evitar renders o cálculos innecesarios.

Implementación conceptual

```
function debounce(callback, delay = 300) {  
  let timeoutId;  
  return (...args) => {  
    clearTimeout(timeoutId);  
    timeoutId = setTimeout(() => callback(...args), delay);  
  };  
}
```

Paso a paso para reciclarlo en otro proyecto

1. Crea la función debounce en tu archivo de utilidades.
2. Envuelve con debounce el callback que hoy se ejecuta demasiado seguido.
3. Ajusta el delay según la experiencia que quieras: 200 a 350 ms suele funcionar bien para búsquedas.
4. Mantén la lógica principal dentro del callback original y deja a debounce solo el control del tiempo.
5. Prueba escribiendo rápido y confirma que la interfaz responde sin saltos.

Donde se usa mejor

- Buscadores de catálogo
- Filtros de tablas
- Autoguardado de formularios
- Validación mientras se escribe

Errores comunes

- Aplicarlo a acciones que deben ser instantáneas, como un botón principal.
- Usar un delay demasiado alto y hacer que la interfaz se sienta lenta.
- Mezclar debounce y lógica de render sin reiniciar correctamente el timeout.

Paso a paso detallado para construirlo desde cero

1. Identifica en tu proyecto el lugar exacto donde necesitas insertar el componente.
Analiza si va en el header, en el main, en un sidebar o en un modal.

2. Crea el HTML mínimo con los ids y clases correctas. Si copias de este banco, asegurate de que no haya ids duplicados en tu página.
3. Copia el CSS que define la apariencia del componente. Presta atención a las variables CSS que controlan colores, espacios, radios y sombras.
4. Si el componente usa animaciones o transiciones, verifica que las propiedades `transition` y `animation` estén definidas en CSS.
5. Copia la clase JavaScript modular o la función normal. Si usas módulos, asegurate de que `script.js` tenga `type="module"` y que las rutas de `import` sean correctas.
6. Inicializa el componente pasando los ids correctos. Si el constructor espera un id y tu HTML tiene otro, cambia el argumento al crear la instancia.
7. Prueba el componente en el navegador. Abre la consola (F12) y verifica que no haya errores rojos.
8. Si hay errores, revisa que los elementos existan en el DOM antes de que JavaScript intente usarlos. Coloca el script al final del body o usa `DOMContentLoaded`.
9. Prueba el componente en diferentes resoluciones: escritorio, tablet y móvil. Usa DevTools para simular distintos anchos de pantalla.
10. Si todo funciona correctamente, guarda una copia de seguridad y documenta cualquier personalización que hayas hecho.

Errores comunes y como solucionarlos

El componente no aparece: revisa que el HTML tenga el id correcto y que el script se cargue después de que el HTML exista. Si el script está en el head, el elemento no existe todavía cuando JavaScript intenta seleccionarlo.

El estilo no se aplica: revisa que las clases CSS del componente estén en `styles.css` y que no haya errores de sintaxis que rompan el resto del archivo.

La interacción no funciona: revisa que los event listeners se conecten a elementos que realmente existen. Si usas `querySelector`, confirma que el selector coincida con el HTML.

El componente se ve mal en móvil: revisa las reglas `@media`. Asegurate de que los anchos fijos, imágenes grandes o grids sin `minmax` no causen scroll horizontal.

Conflictos con otros componentes: si tienes varios componentes con ids o clases similares, puede haber choques. Usa nombres descriptivos y únicos para cada componente.

Checklist de prueba antes de reutilizar

El componente existe en HTML y el id coincide con el usado en JavaScript.

No hay ids duplicados en la página.

La clase visual de estado se agrega y se quita correctamente.

No aparecen errores en la consola del navegador.

La funcionalidad sigue funcionando en pantalla pequeña.

El componente no rompe otros elementos del proyecto.

El CSS no genera scroll horizontal en ninguna resolución.

Los eventos responden al clic, al teclado o al scroll según corresponda.

Que se puede reutilizar directamente

El patron de estructura HTML + CSS + JS de esta funcionalidad se puede adaptar para cualquier componente similar. La logica de seleccion de elementos, manejo de eventos y actualizacion del DOM es un patron universal que se repite en la mayoría de los componentes web.

La forma de organizar el codigo en clases con constructor, init y metodos especificos se puede copiar para crear nuevos componentes. Solo cambias los selectores, los eventos y la logica interna.

Las variables CSS personalizadas se pueden reutilizar en otros componentes para mantener consistencia de colores, espacios y radios.

Variables, clases, ids y atributos que debes cambiar

- ****Elemento**** (Para que sirve): Que debes cambiar
- ****id principal**** (identifica el contenedor del componente): Cambia si ya existe ese id en tu pagina
- ****clases CSS**** (definen la apariencia visual): Cambia si colisionan con otras clases
- ****data-***** (atributos personalizados para JS): Cambia los valores segun tus necesidades
- ****textos visibles**** (etiquetas, titulos, botones): Cambia a los textos de tu proyecto
- ****colores**** (variables CSS o valores hardcodeados): Cambia para que coincidan con tu marca

53. Tarjeta de presentación en Bootstrap

Que hace?

Es una actividad guiada para construir una tarjeta de presentación personal usando la lógica de componentes de Bootstrap como referencia, pero adaptada al estilo visual de este proyecto. La card muestra información personal en una cara frontal (nombre, rol, descripción, enlaces) y una cara trasera con tecnologías o habilidades. El cambio de cara se realiza con un efecto flip 3D al hacer clic en los botones. Todo sin recargar la página.

Objetivo de la actividad

Aprender a estructurar una card con jerarquía visual, aplicar conceptos de Bootstrap (container, row, col, card, badge, btn, shadow) en un contexto personalizado, crear un efecto flip 3D con CSS puro y controlar la interacción con JavaScript. En esta página se resolvió con una interfaz que simula el laboratorio Bootstrap pero con estilos propios.

Estructura general del ejercicio

La interfaz se divide en dos caras dentro de un contenedor 3D. La cara frontal muestra un avatar circular, nombre completo, rol profesional, badges de tecnologías, descripción breve, enlaces de contacto y un botón para ver tecnologías. La cara trasera muestra una lista de tecnologías dominadas, descripción del perfil técnico y un botón para volver al perfil. El contenedor padre aplica perspective y transform-style para lograr el efecto de giro.

HTML mínimo que debes copiar

El contenedor principal usa la clase profile-lab para agrupar toda la actividad. El id profileLab es necesario para que JavaScript pueda seleccionar el contenedor si necesita manipularlo.

El contenedor del flip usa la clase profile-lab__flip y el id profileLabFlip. Este es el elemento que rota. Debe tener position: relative para que las dos caras se posicionen absolutamente dentro de él.

La cara frontal usa profile-lab__face y profile-lab__face--front. Dentro tiene: avatar (profile-lab__avatar con iniciales), nombre (h4), rol (profile-lab__role), badges (profile-lab__badges con span), descripción (p.muted), enlaces (profile-lab__links con a) y botón (showProfileTech con id).

La cara trasera usa profile-lab__face y profile-lab__face--back. Dentro tiene: título (h4), grid de tecnologías (profile-lab__tech-grid con span), descripción (p.muted) y botón (hideProfileTech con id).

Los ids showProfileTech y hideProfileTech son fundamentales porque JavaScript los busca para agregar y quitar la clase is-flipped del contenedor profileLabFlip.

index.html

```
<div class="profile-lab" id="profileLab">
  <div class="profile-lab__flip" id="profileLabFlip">
    <section class="profile-lab__face profile-lab__face--front">
      <div class="profile-lab__avatar">CM</div>
      <h4>Carlos Madero</h4>
```

```

<p class="profile-lab__role">Full Stack Java Developer</p>
<div class="profile-lab__badges">
  <span>Bootstrap</span>
  <span>JavaScript</span>
  <span>Data</span>
</div>
<p class="muted">Card de presentacion personal con una estructura parecida a
un ejercicio Bootstrap, pero adaptada al look de esta guia.</p>
<div class="profile-lab__links">
  <a href="mailto:cmadero08x@gmail.com">Email</a>
  <a href="https://github.com/raydan08x" target="_blank"
rel="nofollow">GitHub</a>
  <a href="https://www.linkedin.com" target="_blank"
rel="nofollow">LinkedIn</a>
</div>
<button type="button" class="primary-button" id="showProfileTech">Ver
tecnologias</button>
</section>
<section class="profile-lab__face profile-lab__face--back">
  <h4>Tecnologias</h4>
  <div class="profile-lab__tech-grid">
    <span>Bootstrap grid</span>
    <span>Card</span>
    <span>Badge</span>
    <span>Buttons</span>
    <span>Shadow</span>
    <span>Responsive</span>
  </div>
  <p class="muted">La actividad te pide comprender que pone Bootstrap y que debe
completar tu CSS personalizado.</p>
  <button type="button" class="secondary-button" id="hideProfileTech">Volver al
perfil</button>
</section>
</div>
</div>

```

CSS del componente

Las reglas de profile-lab usan perspective: 1000px para crear profundidad 3D. Sin esta propiedad, el giro se ve plano y sin efecto de acercamiento. El width y max-width limitan el ancho de la card para que no se desborde en pantallas grandes.

La clase profile-lab__flip usa position: relative, width: 100%, height: 100%, transform-style: preserve-3d y transition: transform 0.6s ease. La propiedad transform-style: preserve-3d es obligatoria para que las dos caras se mantengan en el espacio 3D mientras giran. Sin ella, el navegador aplanará el giro y se ve mal. El transition controla la duración del giro.

La clase is-flipped aplica transform: rotateY(180deg). Cuando JavaScript agrega esta clase al contenedor, la card gira 180 grados en el eje Y, mostrando la cara trasera.

La clase profile-lab__face usa position: absolute, inset: 0, backface-visibility: hidden y border-radius. La propiedad backface-visibility: hidden es crítica porque evita que se vea la cara trasera al revés cuando la frontal está visible. Sin ella, el texto de ambas caras se superpone y se lee mal.

La cara trasera `profile-lab__face--back` usa `transform: rotateY(180deg)` para que ya este orientada al revés desde el inicio. Cuando el contenedor gira, esta cara queda frontal.

El avatar `profile-lab__avatar` usa `width`, `height`, `border-radius: 50%`, `display: grid`, `place-items: center` y un gradiente de fondo. Esto crea un círculo perfecto con iniciales centradas.

Los badges `profile-lab__badges span` usan `padding`, `border-radius`, `border` y `background` para parecerse a los badges de Bootstrap. El `display: flex` con `gap` alinea los badges horizontalmente.

Los enlaces `profile-lab__links a` usan `padding`, `border-radius`, `border` y `text-decoration: none` para parecer botones secundarios. El `hover` cambia el fondo y el borde para dar retroalimentación.

La grid de tecnologías `profile-lab__tech-grid` usa `display: grid` con `grid-template-columns: repeat(3, 1fr)` y `gap`. Cada `span` tiene `padding`, `border-radius`, `background` y `border` para parecerse a ítems de conocimiento.

styles.css

```
.profile-lab {
  perspective: 1000px;
  width: 100%;
  max-width: 400px;
  margin: 0 auto;
}
.profile-lab__flip {
  position: relative;
  width: 100%;
  height: 100%;
  transform-style: preserve-3d;
  transition: transform 0.6s ease;
}
.profile-lab__flip.is-flipped {
  transform: rotateY(180deg);
}
.profile-lab__face {
  position: absolute;
  inset: 0;
  backface-visibility: hidden;
  border-radius: 20px;
  padding: 24px;
  background: rgba(255, 255, 255, 0.08);
  border: 1px solid rgba(255, 255, 255, 0.22);
  display: grid;
  gap: 16px;
  align-content: start;
}
.profile-lab__face--back {
  transform: rotateY(180deg);
}
.profile-lab__avatar {
  width: 80px;
  height: 80px;
```

```
border-radius: 50%;
background: linear-gradient(135deg, #2563eb, #7c3aed);
color: #ffffff;
display: grid;
place-items: center;
font-size: 28px;
font-weight: 800;
margin: 0 auto;
}
.profile-lab__role {
  font-size: 14px;
  color: var(--muted);
  margin: 0;
}
.profile-lab__badges {
  display: flex;
  justify-content: center;
  gap: 8px;
  flex-wrap: wrap;
}
.profile-lab__badges span {
  padding: 4px 10px;
  border-radius: 999px;
  background: rgba(255, 255, 255, 0.1);
  border: 1px solid rgba(255, 255, 255, 0.22);
  font-size: 12px;
  font-weight: 700;
}
.profile-lab__links {
  display: flex;
  justify-content: center;
  gap: 10px;
  flex-wrap: wrap;
}
.profile-lab__links a {
  padding: 8px 14px;
  border-radius: 10px;
  border: 1px solid rgba(255, 255, 255, 0.22);
  background: rgba(255, 255, 255, 0.08);
  color: inherit;
  text-decoration: none;
  font-size: 13px;
  font-weight: 700;
  transition: background var(--transition), border-color var(--transition);
}
.profile-lab__links a:hover {
  background: rgba(255, 255, 255, 0.14);
  border-color: var(--primary);
}
.profile-lab__tech-grid {
  display: grid;
  grid-template-columns: repeat(3, 1fr);
  gap: 10px;
}
.profile-lab__tech-grid span {
```

```
padding: 8px;
border-radius: 10px;
background: rgba(255, 255, 255, 0.08);
border: 1px solid rgba(255, 255, 255, 0.22);
font-size: 12px;
font-weight: 700;
text-align: center;
}
```

JavaScript modular con export/import

Usa esta version si tu HTML carga el script con type="module".

La clase ProfilePresentationActivity busca el contenedor del flip con getElementById('profileLabFlip'). Tambien busca los dos botones: showProfileTech y hideProfileTech. Si alguno de estos elementos no existe, la clase no se inicializa.

En el constructor se guardan las referencias en this. El metodo init() conecta los eventos. Cuando se hace clic en showProfileTech, se agrega la clase is-flipped al contenedor. Cuando se hace clic en hideProfileTech, se quita la clase is-flipped.

La logica es intencionalmente simple porque la mayor parte del trabajo visual la hace CSS. JavaScript solo actua como interruptor: agrega o quita una clase. Este patron es muy eficiente porque delega la animacion al navegador a traves de CSS, en lugar de calcular frames con JavaScript.

La propiedad classList.toggle podria usarse, pero en este caso se usa classList.add y classList.remove para ser explicito. Si usaras toggle en ambos botones, podrias terminar con estados inesperados si el usuario hace clic dos veces en el mismo boton.

funciones.js + script.js

```
export class ProfilePresentationActivity {
  constructor() {
    this.flip = document.getElementById('profileLabFlip');
    this.showButton = document.getElementById('showProfileTech');
    this.hideButton = document.getElementById('hideProfileTech');
    if (this.flip && this.showButton && this.hideButton) {
      this.init();
    }
  }

  init() {
    this.showButton.addEventListener('click', () => {
      this.flip.classList.add('is-flipped');
    });
    this.hideButton.addEventListener('click', () => {
      this.flip.classList.remove('is-flipped');
    });
  }
}

// script.js
import { ProfilePresentationActivity } from './funciones.js';
new ProfilePresentationActivity();
```

JavaScript normal sin modulos

Usa esta version si tu HTML carga el script sin type="module".

La logica es exactamente la misma que la version modular. La diferencia es que no hay export/import. Todo vive en el mismo archivo. La clase ProfilePresentationActivity no lleva export.

Esta version es util para ejercicios academicos, paginas simples o cuando no quieres configurar modulos. Solo tienes que asegurarte de que este script se cargue despues de que el HTML exista, idealmente al final del body o con DOMContentLoaded.

script.js

```
class ProfilePresentationActivity {
  constructor() {
    this.flip = document.getElementById('profileLabFlip');
    this.showButton = document.getElementById('showProfileTech');
    this.hideButton = document.getElementById('hideProfileTech');
    if (this.flip && this.showButton && this.hideButton) {
      this.init();
    }
  }

  init() {
    this.showButton.addEventListener('click', () => {
      this.flip.classList.add('is-flipped');
    });
    this.hideButton.addEventListener('click', () => {
      this.flip.classList.remove('is-flipped');
    });
  }
}

new ProfilePresentationActivity();
```

Errores comunes y como solucionarlos

El giro se ve plano o sin profundidad: si falta perspective en el contenedor padre, el giro parece una simple transparencia en lugar de una rotacion 3D. Asegurate de que profile-lab tenga perspective: 1000px o similar. Valores entre 800px y 1200px suelen funcionar bien.

El texto se superpone al girar: si falta backface-visibility: hidden en las caras, ambas caras son visibles al mismo tiempo y el texto se lee invertido o superpuesto. Asegurate de que profile-lab__face tenga backface-visibility: hidden.

La cara trasera se ve al reves: si olvidas transform: rotateY(180deg) en profile-lab__face--back, la cara trasera aparece invertida cuando el contenedor gira. Esta regla es obligatoria porque la cara trasera debe estar orientada al reves desde el inicio para que al girar el contenedor quede frontal.

El giro no funciona: si los ids showProfileTech o hideProfileTech no coinciden con los del HTML, los botones no encuentran sus listeners. Revisa que los ids en el HTML y en JavaScript sean exactamente iguales. Tambien verifica que el script se cargue despues del HTML.

La card se corta en movil: si el contenido de la cara trasera es mas alto que el de la frontal, el contenedor puede cortarse al girar. Asegurate de que ambas caras tengan contenido similar en altura, o usa min-height en el contenedor para reservar espacio suficiente.

Los badges no se alinean: si usas display: flex sin justify-content: center o flex-wrap: wrap, los badges pueden quedar desalineados o desbordarse. Revisa que profile-lab__badges tenga ambas propiedades.

Checklist de prueba antes de reutilizar

El componente existe en HTML y los ids coinciden exactamente con los usados en JavaScript.

No hay ids duplicados en la pagina porque getElementById solo encuentra el primero.

El contenedor padre tiene perspective para el efecto 3D.

Las caras tienen backface-visibility: hidden.

La cara trasera tiene transform: rotateY(180deg).

El giro funciona al hacer clic en ambos botones.

El contenido se lee bien en tema oscuro y claro.

Los badges y enlaces se alinean correctamente.

La card no se corta en pantallas pequenas.

No hay errores en la consola del navegador.

Mejoras opcionales

Conectar el componente con un Toast para avisar cuando se cambia de cara.

Agregar una tercera cara con proyectos o portafolio.

Guardar el estado actual (cara frontal o trasera) en localStorage para recordarlo al recargar.

Permitir girar la card al hacer clic en cualquier parte de ella, no solo en los botones.

Agregar animaciones suaves al pasar el mouse sobre los badges o enlaces.

Mejorar accesibilidad con aria-label en los botones y aria-live para anunciar el cambio de cara a lectores de pantalla.

Reemplazar el avatar de iniciales por una imagen real usando img con border-radius: 50%.

Que se puede reutilizar directamente

El componente profile-lab se puede llevar a cualquier proyecto que necesite una tarjeta de presentacion con dos caras. El patron de flip con CSS 3D es facil de adaptar para tarjetas de producto, tarjetas de servicio, tarjetas de equipo o tarjetas de certificacion.

La estructura de avatar circular con iniciales se puede reemplazar por una imagen real con img y border-radius: 50%. El gradiente de fondo del avatar se puede cambiar por cualquier color o imagen.

El sistema de badges con display: flex, justify-content: center y flex-wrap: wrap se puede reutilizar para cualquier lista de etiquetas: tecnologias, habilidades, categorias, roles,

estados o tags.

Los enlaces estilizados como botones secundarios con hover se pueden reutilizar para cualquier grupo de acciones: contacto, redes sociales, descargas, compartir o navegacion interna.

La grid de tecnologías con grid-template-columns: repeat(3, 1fr) se puede adaptar para cualquier lista de items: herramientas, skills, proyectos, certificaciones o estadísticas.

El efecto flip con perspective, transform-style: preserve-3d y backface-visibility: hidden es un patron CSS puro que funciona en cualquier navegador moderno sin necesidad de librerías.

Variables, clases, ids y atributos que debes cambiar

- ****Elemento**** (Para que sirve): Que debes cambiar
- ****profileLab**** (id del contenedor principal): Cambia si ya tienes otro id igual
- ****profileLabFlip**** (id del contenedor que rota): Cambia si ya existe ese id en la pagina
- ****showProfileTech**** (id del boton para mostrar tecnologías): Cambia si ya existe ese id
- ****hideProfileTech**** (id del boton para volver al perfil): Cambia si ya existe ese id
- ****profile-lab**** (clase del contenedor principal): Cambia si colisiona con otra clase
- ****profile-lab__flip**** (clase del contenedor 3D): Cambia si colisiona con otra clase
- ****profile-lab__face**** (clase base de las caras): No cambies el nombre porque ya existe en el proyecto
- ****profile-lab__face--front**** (clase de la cara frontal): Cambia si quieres otro nombre
- ****profile-lab__face--back**** (clase de la cara trasera): Cambia si quieres otro nombre
- ****profile-lab__avatar**** (clase del avatar circular): Cambia si colisiona con otra clase
- ****profile-lab__role**** (clase del rol profesional): Cambia si colisiona con otra clase
- ****profile-lab__badges**** (clase de los badges): Cambia si colisiona con otra clase
- ****profile-lab__links**** (clase de los enlaces): Cambia si colisiona con otra clase
- ****profile-lab__tech-grid**** (clase de la grid de tecnologías): Cambia si colisiona con otra clase
- ****is-flipped**** (clase que activa el giro): No cambies el nombre porque ya existe en el proyecto
- ****CM**** (iniciales del avatar): Cambia por las iniciales de la persona
- ****Carlos Madero**** (nombre completo): Cambia por el nombre real
- ****Full Stack Java Developer**** (rol profesional): Cambia por el rol real
- ****Bootstrap**** (texto del badge): Cambia por la tecnología real
- ****JavaScript**** (texto del badge): Cambia por la tecnología real
- ****Data**** (texto del badge): Cambia por la tecnología real
- ****Email / GitHub / LinkedIn**** (textos de los enlaces): Cambia por las redes sociales reales
- ****Ver tecnologías**** (texto del boton frontal): Cambia por el texto que necesites

- ****Volver al perfil**** (texto del boton trasero): Cambia por el texto que necesites

54. Carrito de compras DOM

Que hace?

Es una actividad guiada para practicar manipulacion del DOM con JavaScript puro. Simula una mini tienda donde puedes agregar productos, actualizar un contador visual (badge), calcular el precio total acumulado, eliminar items individuales y vaciar el carrito completo. Todo sin recargar la pagina.

Objetivo de la actividad

Aprender a conectar botones con data-attributes, crear listas dinamicas en el DOM y mantener calculos en tiempo real. En esta pagina se resolvio con una interfaz compacta que sigue el estilo del proyecto.

Estructura general del ejercicio

La interfaz se divide en tres zonas claras. Una barra superior muestra el nombre de la tienda y el badge del carrito. El grid de productos contiene cards con nombre, precio y boton Agregar. La columna del carrito muestra la lista de items agregados, el total acumulado y un boton para vaciar todo.

HTML minimo que debes copiar

El contenedor principal usa la clase shop-lab para agrupar todo el componente.

La barra superior shop-lab__topbar muestra el titulo Mini tienda y el badge shop-lab__badge con id shopBadge. Este badge es el numero circular que cambia cada vez que agregas un producto. El id es necesario para que JavaScript pueda seleccionarlo y actualizar su texto.

El grid shop-lab__grid usa dos columnas para mostrar los productos. Cada article con clase shop-product representa un producto. Dentro hay un boton con clase shop-add-button y dos atributos data-name y data-price. Esos atributos son fundamentales porque JavaScript los lee para saber que nombre y precio tiene cada producto sin tener que buscar texto dentro del HTML.

La seccion del carrito usa shop-cart. Tiene un encabezado shop-cart__head con el titulo Carrito y el boton Vaciar con id clearShopCart. El mensaje vacio usa shop-cart__empty con id shopEmptyMessage. Este parrafo solo se muestra cuando no hay items. La lista shop-cart__list con id shopCartList es el ul donde JavaScript inserta cada item nuevo. El total usa shop-cart__total con shopCartTotal para actualizar el precio acumulado.

index.html

```
<div class="shop-lab">
  <div class="shop-lab__topbar">
    <strong>Mini tienda</strong>
    <span class="shop-lab__badge" id="shopBadge">0</span>
  </div>

  <div class="shop-lab__grid">
    <article class="shop-product">
      <h4>Pack IPA</h4>
      <p class="muted">$18,000</p>
    </article>
  </div>
</div>
```

```

        <button type="button" class="primary-button shop-add-button"
            data-name="Pack IPA" data-price="18000">Agregar</button>
    </article>

    <article class="shop-product">
        <h4>Stout Reserva</h4>
        <p class="muted">$16,500</p>
        <button type="button" class="primary-button shop-add-button"
            data-name="Stout Reserva" data-price="16500">Agregar</button>
    </article>
</div>

<div class="shop-cart">
    <div class="shop-cart__head">
        <h4>Carrito</h4>
        <button type="button" class="danger-button" id="clearShopCart">Vaciar</button>
    </div>
    <p class="muted shop-cart__empty" id="shopEmptyMessage">Tu carrito esta
vacio.</p>
    <ul class="shop-cart__list" id="shopCartList"></ul>
    <div class="shop-cart__total">
        <span>Total</span>
        <strong id="shopCartTotal">$0</strong>
    </div>
</div>
</div>

```

CSS del componente

Las reglas de shop-lab usan display: grid para separar las zonas con gap. La barra superior usa display: flex con justify-content: space-between para que el titulo quede a la izquierda y el badge a la derecha. El fondo usa rgba(255,255,255,0.08) para mantener el estilo glass del proyecto sin tapar lo de atras.

El badge usa border-radius: 999px para hacerlo completamente circular. El fondo es un gradiente de azul a morado para que resalte visualmente como un contador activo.

El grid de productos usa grid-template-columns: repeat(2, minmax(0, 1fr)) para mostrar dos columnas. Cada producto tiene padding y bordes redondeados para mantener la coherencia visual.

Los items del carrito usan shop-cart__item con display: flex y justify-content: space-between para que el nombre y precio queden a la izquierda y el boton Eliminar a la derecha. Este layout es clave para que la lista se vea ordenada sin necesidad de tablas.

styles.css

```

.shop-lab {
    display: grid;
    gap: 18px;
}
.shop-lab__topbar {
    display: flex;
    align-items: center;
    justify-content: space-between;
    gap: 12px;
}

```

```
padding: 14px 16px;
border-radius: 16px;
background: rgba(255, 255, 255, 0.08);
border: 1px solid rgba(255, 255, 255, 0.22);
}
.shop-lab__badge {
  min-width: 34px;
  height: 34px;
  display: inline-grid;
  place-items: center;
  padding: 0 10px;
  border-radius: 999px;
  color: #ffffff;
  font-weight: 800;
  background: linear-gradient(135deg, #2563eb, #7c3aed);
}
.shop-lab__grid {
  display: grid;
  grid-template-columns: repeat(2, minmax(0, 1fr));
  gap: 12px;
}
.shop-product {
  padding: 16px;
  border-radius: 16px;
  background: rgba(255, 255, 255, 0.08);
  border: 1px solid rgba(255, 255, 255, 0.22);
}
.shop-cart {
  padding: 16px;
  border-radius: 16px;
  background: rgba(255, 255, 255, 0.08);
  border: 1px solid rgba(255, 255, 255, 0.22);
  display: grid;
  gap: 12px;
}
.shop-cart__head {
  display: flex;
  align-items: center;
  justify-content: space-between;
  gap: 12px;
}
.shop-cart__list {
  list-style: none;
  padding: 0;
  margin: 0;
  display: grid;
  gap: 10px;
}
.shop-cart__item {
  display: flex;
  align-items: center;
  justify-content: space-between;
  gap: 12px;
  padding: 12px;
  border-radius: 10px;
```

```

    background: rgba(255, 255, 255, 0.08);
    border: 1px solid rgba(255, 255, 255, 0.22);
  }
  .shop-cart__total {
    display: flex;
    align-items: center;
    justify-content: space-between;
    gap: 12px;
  }

```

JavaScript modular con export/import

Usa esta version si tu HTML carga el script con type="module".

Los helpers \$ y \$\$ abrevian querySelector y querySelectorAll. Se usan para no repetir document.querySelector cada vez que necesitas un elemento. El helper \$ busca el primer match. El helper \$\$ convierte los resultados en un array con [...] para poder usar metodos como forEach directamente.

La clase ShoppingCartActivity recibe toast como parametro opcional. El toast se usa para mostrar mensajes tipo Producto agregado al carrito. Si no lo necesitas, puedes omitirlo.

En el constructor se guardan las referencias a los elementos del DOM. Se usa querySelectorAll para obtener todos los botones .shop-add-button. Se usa getElementById para la lista, el badge, el total, el mensaje vacio y el boton de vaciar. Estas referencias se guardan en this para que todos los metodos de la clase puedan usarlas sin volver a buscarlas.

Las variables itemCount y totalAmount empiezan en 0. itemCount cuenta cuantos productos hay en el carrito. totalAmount suma los precios de todos los productos agregados.

El metodo init conecta los eventos. Recorre cada boton con forEach y agrega un listener de click. Cuando haces click, lee button.dataset.name y button.dataset.price. Esos valores vienen directamente de los atributos data-* del HTML. El precio se convierte a numero con Number() para poder sumarlo despues.

El boton de vaciar tambien tiene un listener que llama a this.clear().

El metodo renderSummary() se ejecuta al final del init para que el badge muestre 0, el total muestre \$0 y el mensaje vacio sea visible desde el inicio.

El metodo addItem recibe name y price. Crea un li con createElement. Usa innerHTML para insertar la estructura del item: un div con el nombre y precio, y un boton Eliminar. El boton usa clase danger-button para el color rojo.

El listener del boton Eliminar se crea dentro de addItem. Busca el boton dentro del item con \$('button', item). El signo de interrogacion ? es optional chaining: si no encuentra el boton, no falla. El listener llama a removeItem pasando el item completo y el precio.

Despues de agregar el item al DOM con appendChild, se suma 1 al contador y el precio al total. Luego se llama renderSummary() para actualizar la UI. Si existe el toast, muestra un mensaje de confirmacion.

El metodo removeItem usa item.remove() para eliminar el elemento del DOM. Es una de las

formas mas directas de quitar un nodo. Resta 1 al contador y el precio al total. `Math.max(0, ...)` evita que los numeros queden negativos si algo sale mal. Luego `renderSummary()` actualiza todo.

El metodo `clear()` selecciona todos los `li` de la lista con `$$('li', this.list)`. `forEach` los elimina uno por uno. Resetea los contadores a 0 y actualiza la UI. Si hay toast, muestra Carrito vaciado.

El metodo `renderSummary()` actualiza tres cosas: el badge con la cantidad, el total con el precio formateado, y la visibilidad del mensaje vacio. Si hay items, oculta el mensaje. Si no hay, lo muestra. La propiedad `style.display` controla esto directamente.

El metodo `formatPrice` usa `toLocaleString('es-CO')` para mostrar los precios con puntos de miles. El valor se convierte en algo como \$18.000 en lugar de \$18000.

funciones.js + script.js

```
export const $ = (selector, parent = document) => parent.querySelector(selector);
export const $$ = (selector, parent = document) =>
[...parent.querySelectorAll(selector)];
```

```
export class ShoppingCartActivity {
  constructor(toast) {
    this.toast = toast;
    this.buttons = $$('.shop-add-button');
    this.list = $('#shopCartList');
    this.badge = $('#shopBadge');
    this.total = $('#shopCartTotal');
    this.emptyMessage = $('#shopEmptyMessage');
    this.clearButton = $('#clearShopCart');
    this.itemsCount = 0;
    this.totalAmount = 0;
    if (this.buttons.length && this.list && this.badge && this.total &&
this.emptyMessage && this.clearButton) {
      this.init();
    }
  }
}
```

```
init() {
  this.buttons.forEach((button) => {
    button.addEventListener('click', () => {
      this.addItem(button.dataset.name, Number(button.dataset.price));
    });
  });
  this.clearButton.addEventListener('click', () => this.clear());
  this.renderSummary();
}
```

```
addItem(name, price) {
  const item = document.createElement('li');
  item.className = 'shop-cart__item';
  item.innerHTML = '
    <div>
      <strong>${name}</strong>
      <p class="muted">${this.formatPrice(price)}</p>
```

```

        </div>
        <button type="button" class="danger-button">X</button>
    ';
    $('button', item)?.addEventListener('click', () => this.removeItem(item,
price));
    this.list.appendChild(item);
    this.itemsCount += 1;
    this.totalAmount += price;
    this.renderSummary();
    this.toast?.show(`${name} agregado al carrito.`);
}

removeItem(item, price) {
    item.remove();
    this.itemsCount = Math.max(0, this.itemsCount - 1);
    this.totalAmount = Math.max(0, this.totalAmount - price);
    this.renderSummary();
}

clear() {
    $('li', this.list).forEach((item) => item.remove());
    this.itemsCount = 0;
    this.totalAmount = 0;
    this.renderSummary();
    this.toast?.show('Carrito vaciado.`);
}

renderSummary() {
    this.badge.textContent = String(this.itemsCount);
    this.total.textContent = this.formatPrice(this.totalAmount);
    this.emptyMessage.style.display = this.itemsCount ? 'none' : 'block';
}

formatPrice(value) {
    return `$$${value.toLocaleString('es-CO')}`;
}
}

// script.js
import { ShoppingCartActivity } from './funciones.js';
const cart = new ShoppingCartActivity();

```

JavaScript normal sin modulos

Usa esta version si tu HTML carga el script sin type="module".

La logica es exactamente la misma que la version modular. La diferencia es que no hay export/import. Todo vive en el mismo archivo. Los helpers \$ y \$\$ se definen como constantes normales. La clase ShoppingCartActivity no lleva export.

Esta version es util para ejercicios academicos, paginas simples o cuando no quieres configurar modulos. Solo tienes que asegurarte de que este script se cargue despues de que el HTML exista, idealmente al final del body o con DOMContentLoaded.

script.js

```
const $ = (selector, parent = document) => parent.querySelector(selector);
const $$ = (selector, parent = document) => [...parent.querySelectorAll(selector)];
```

```
class ShoppingCartActivity {
  constructor(toast) {
    this.toast = toast;
    this.buttons = $$('.shop-add-button');
    this.list = $('#shopCartList');
    this.badge = $('#shopBadge');
    this.total = $('#shopCartTotal');
    this.emptyMessage = $('#shopEmptyMessage');
    this.clearButton = $('#clearShopCart');
    this.itemsCount = 0;
    this.totalAmount = 0;
    if (this.buttons.length && this.list && this.badge && this.total &&
this.emptyMessage && this.clearButton) {
      this.init();
    }
  }

  init() {
    this.buttons.forEach((button) => {
      button.addEventListener('click', () => {
        this.addItem(button.dataset.name, Number(button.dataset.price));
      });
    });
    this.clearButton.addEventListener('click', () => this.clear());
    this.renderSummary();
  }

  addItem(name, price) {
    const item = document.createElement('li');
    item.className = 'shop-cart__item';
    item.innerHTML = '
      <div>
        <strong>${name}</strong>
        <p class="muted">${this.formatPrice(price)}</p>
      </div>
      <button type="button" class="danger-button">X</button>
    ';
    $('button', item)?.addEventListener('click', () => this.removeItem(item,
price));
    this.list.appendChild(item);
    this.itemsCount += 1;
    this.totalAmount += price;
    this.renderSummary();
    this.toast?.show(`${name} agregado al carrito.`);
  }

  removeItem(item, price) {
    item.remove();
    this.itemsCount = Math.max(0, this.itemsCount - 1);
    this.totalAmount = Math.max(0, this.totalAmount - price);
    this.renderSummary();
  }
}
```

```

clear() {
  $$('li', this.list).forEach((item) => item.remove());
  this.itemsCount = 0;
  this.totalAmount = 0;
  this.renderSummary();
  this.toast?.show('Carrito vaciado.');
```

```

}

renderSummary() {
  this.badge.textContent = String(this.itemsCount);
  this.total.textContent = this.formatPrice(this.totalAmount);
  this.emptyMessage.style.display = this.itemsCount ? 'none' : 'block';
}

formatPrice(value) {
  return '$${value.toLocaleString('es-CO')}}';
}
}

const cart = new ShoppingCartActivity();
```

Errores comunes y como solucionarlos

No cambia el total: el precio se lee como texto en lugar de numero. Si olvidas `Number()` al leer `dataset.price`, el signo + se convierte en concatenacion de strings y terminas con \$018000 en lugar de \$18.000. Siempre usa `Number()` para convertir el precio a valor numerico antes de sumar.

El boton Eliminar no funciona: el listener debe crearse dentro del metodo `addItem`, justo despues de crear el item con `createElement`. Si lo creas fuera o en el constructor, no existe el boton todavia. La forma correcta es buscar el boton dentro del item nuevo con `$( 'button', item)` y conectarle el listener en ese momento.

El badge queda descuadrado: si solo actualizas el badge al agregar pero no al eliminar o vaciar, el numero queda desactualizado. Asegurate de llamar `renderSummary()` al final de `addItem`, `removeItem` y `clear()` para que el badge siempre refleje la cantidad real.

El mensaje vacio no desaparece: si olvidas manejar la visibilidad del mensaje vacio en `renderSummary()`, el parrafo Tu carrito esta vacio sigue visible aunque haya items. La propiedad `style.display` con valor `'none'` o `'block'` controla esto. Revisa que `emptyMessage` exista y que `renderSummary()` se ejecute despues de cada cambio.

Checklist de prueba antes de reutilizar

El componente existe en HTML y los ids coinciden exactamente con los usados en JavaScript.

No hay ids duplicados en la pagina porque `getElementById` solo encuentra el primero.

Los botones Agregar tienen `data-name` y `data-price` con valores correctos.

El mensaje vacio aparece y desaparece segun haya o no items.

No hay errores en la consola del navegador.

La funcionalidad sigue funcionando en pantalla pequena.

El componente no rompe otros elementos del proyecto.

Mejoras opcionales

Conectar el componente con un Toast para avisar cada accion: agregar, eliminar o vaciar.

Guardar el carrito en localStorage para que no se pierda al recargar la pagina.

Agregar validaciones para evitar agregar el mismo producto dos veces o permitir cantidades.

Mejorar accesibilidad con aria-label en los botones y aria-live en la lista para lectores de pantalla.

Agregar animaciones suaves al agregar o eliminar items para que la UI se sienta mas viva.

Que se puede reutilizar directamente

El componente shop-lab se puede llevar a cualquier proyecto que necesite una mini tienda, carrito de compras, lista de servicios o cotizador. El patron de botones con data-name y data-price es facil de adaptar para cualquier catalogo: productos, servicios, reservas, paquetes o suscripciones.

El sistema de badge con contador visual se puede reutilizar para cualquier contador: notificaciones, mensajes, tareas, items seleccionados, pasos completados o alertas. El gradiente de azul a morado se puede cambiar por cualquier otro color.

El grid de productos con grid-template-columns: repeat(2, minmax(0, 1fr)) se puede adaptar para cualquier lista de items: cards de producto, cards de servicio, cards de equipo o cards de proyecto. Cambia el numero de columnas segun el ancho disponible.

El layout del carrito con flex y justify-content: space-between se puede reutilizar para cualquier lista de items con acciones: lista de tareas, lista de compras, lista de favoritos, lista de pendientes o lista de contactos.

El metodo de formateo de precios con toLocaleString('es-CO') se puede adaptar para cualquier moneda: cambia 'es-CO' por 'en-US', 'es-ES', 'pt-BR' o cualquier otro locale. Cambia el simbolo \$ por el de la moneda que necesites.

El sistema de vaciar carrito con forEach y remove() es un patron reutilizable para cualquier accion de limpieza: vaciar lista, borrar seleccion, limpiar filtros o resetear formulario.

Variables, clases, ids y atributos que debes cambiar

- ****Elemento**** (Para que sirve): Que debes cambiar
- ****shopLab**** (id del contenedor principal): Cambia si ya tienes otro id igual
- ****shopBadge**** (id del badge del carrito): Cambia si ya existe ese id
- ****shopCartList**** (id de la lista de items): Cambia si ya existe ese id
- ****shopCartTotal**** (id del total acumulado): Cambia si ya existe ese id
- ****shopEmptyMessage**** (id del mensaje de carrito vacio): Cambia si ya existe ese id
- ****clearShopCart**** (id del boton vaciar): Cambia si ya existe ese id
- ****shop-lab**** (clase del contenedor principal): Cambia si colisiona con otra clase
- ****shop-lab__topbar**** (clase de la barra superior): Cambia si colisiona con otra clase
- ****shop-lab__badge**** (clase del badge circular): Cambia si colisiona con otra clase

- **shop-lab__grid** (clase del grid de productos): Cambia si colisiona con otra clase
- **shop-product** (clase de cada producto): Cambia si colisiona con otra clase
- **shop-add-button** (clase de los botones agregar): Cambia si colisiona con otra clase
- **shop-cart** (clase del contenedor del carrito): Cambia si colisiona con otra clase
- **shop-cart__head** (clase del encabezado del carrito): Cambia si colisiona con otra clase
- **shop-cart__list** (clase de la lista de items): Cambia si colisiona con otra clase
- **shop-cart__item** (clase de cada item en la lista): Cambia si colisiona con otra clase
- **shop-cart__total** (clase del total): Cambia si colisiona con otra clase
- **shop-cart__empty** (clase del mensaje vacio): Cambia si colisiona con otra clase
- **itemsCount** (variable interna de cantidad): Cambia el nombre si prefieres otro
- **totalAmount** (variable interna de precio total): Cambia el nombre si prefieres otro
- **data-name** (atributo del boton con nombre del producto): Cambia los valores por los productos reales
- **data-price** (atributo del boton con precio del producto): Cambia los valores por los precios reales
- **Pack IPA** (nombre del producto ejemplo): Cambia por los productos reales
- **\$18,000** (precio del producto ejemplo): Cambia por los precios reales
- **Mini tienda** (titulo de la tienda): Cambia por el nombre de tu tienda
- **Carrito** (titulo del carrito): Cambia por el nombre de tu lista
- **Vaciar** (texto del boton vaciar): Cambia por el texto que necesites
- **X** (texto del boton eliminar): Cambia por icono o texto que prefieras
- **Total** (texto del total): Cambia por el texto que necesites

55. Footer reutilizable

Que hace?

Es un componente estructural que se coloca al final de toda pagina web. Proporciona informacion de marca, enlaces de navegacion rapidos, datos de contacto, redes sociales y una barra de copyright. Es fundamental para cualquier sitio profesional porque cierra la jerarquia visual, mejora la navegacion y da credibilidad al proyecto.

Objetivo del componente

Crear un pie de pagina que funcione como cierre coherente de la interfaz, que sea totalmente responsive y que se pueda adaptar a cualquier proyecto simplemente cambiando textos, enlaces y colores. En esta pagina se resolvió con un estilo glass sutil que combina con el tema del proyecto.

Estructura general del componente

La interfaz se divide en tres zonas principales. La zona superior usa una grid de tres columnas: marca con descripcion, enlaces de navegacion y datos de contacto. La zona inferior es una barra separada por borde que muestra copyright y una etiqueta informativa. Todo esta contenido dentro de un footer con fondo semitransparente y borde superior.

HTML minimo que debes copiar

El contenedor principal usa la etiqueta semantica footer con clase site-footer y id siteFooter. El id es opcional si no necesitas manipularlo con JavaScript, pero es util para aplicar estilos especificos o animaciones de entrada.

La grid superior usa site-footer__grid con display: grid. En escritorio usa tres columnas: 2fr para la marca y 1fr para cada una de las otras dos columnas. Esto da mas espacio a la descripcion y distribuye equitativamente los enlaces.

La columna de marca site-footer__brand contiene el mismo logo o marca que la navbar. El span brand__mark usa el mismo estilo circular que el header para mantener coherencia visual. El parrafo muted usa la clase existente del proyecto para descripciones secundarias.

La columna de enlaces site-footer__links usa un elemento nav semantico con aria-label para accesibilidad. Dentro hay enlaces a que apuntan a secciones internas con hash o rutas externas. Los enlaces internos se conectan automaticamente con el router de secciones si tu proyecto usa el patron data-section.

La columna de contacto site-footer__contact usa enlaces a para mailto, GitHub y LinkedIn. El atributo target=_blank y rel=noreferrer son obligatorios para enlaces externos.

La barra inferior site-footer__bottom usa dos span: uno para copyright con el simbolo y ano, y otro para una etiqueta informativa como el numero de funcionalidades o version del proyecto.

index.html

```
<footer class="site-footer" id="siteFooter">
  <div class="site-footer__grid">
    <div class="site-footer__brand">
```

```

    <span class="brand__mark">CW</span>
    <strong>Componentes Web</strong>
    <p class="muted">Banco de componentes reutilizables HTML + CSS + JS para
proyectos profesionales.</p>
</div>
<div class="site-footer__links">
    <strong>Secciones</strong>
    <nav aria-label="Footer navegacion">
        <a href="#inicio" data-section="inicio">Inicio</a>
        <a href="#componentes" data-section="componentes">Componentes</a>
        <a href="#formularios" data-section="formularios">Formularios</a>
        <a href="#datos" data-section="datos">Datos</a>
        <a href="#utilidades" data-section="utilidades">Utilidades</a>
        <a href="#guia" data-section="guia">Guia</a>
    </nav>
</div>
<div class="site-footer__contact">
    <strong>Contacto</strong>
    <a href="mailto:cmadero08x@gmail.com">cmadero08x@gmail.com</a>
    <a href="https://github.com/raydan08x" target="_blank"
rel="noreferrer">GitHub</a>
    <a href="https://www.linkedin.com" target="_blank"
rel="noreferrer">LinkedIn</a>
</div>
</div>
<div class="site-footer__bottom">
    <span>2026 Carlos Madero. Todos los derechos reservados.</span>
    <span>Kit Web Reutilizable 55 funcionalidades documentadas</span>
</div>
</footer>

```

CSS del componente

Las reglas de site-footer usan margin-top: 40px para separar el footer del contenido principal. El padding superior es mayor que el inferior para dar espacio visual. El background usa rgba(255, 255, 255, 0.04) que es un gris muy sutil que contrasta con el fondo de la pagina sin tapar el efecto glass. El border-top usa var(--border) para mantener coherencia con el resto del proyecto.

La clase site-footer__grid usa display: grid con max-width: 1120px y margin: 0 auto para centrar el contenido. El gap: 40px separa las columnas sin necesidad de padding interno. La propiedad grid-template-columns: 2fr 1fr 1fr distribuye el espacio: la marca ocupa el doble de ancho que cada columna de enlaces.

La marca site-footer__brand reutiliza la clase brand__mark del header para el circulo del logo. El strong usa font-size: 18px para que el nombre del proyecto sea visible pero no compita con el contenido principal.

Los titulos de columna site-footer__links strong y site-footer__contact strong usan text-transform: uppercase, letter-spacing: 0.04em y font-size: 14px para parecerse a subtítulos de footer profesionales. El color usa var(--text) para mantener legibilidad.

La navegacion de enlaces site-footer__links nav usa display: grid con gap: 8px para apilar los enlaces verticalmente. Cada enlace usa display: block para ocupar toda la anchura de la

columna, facilitando el clic en dispositivos táctiles.

Los enlaces a comparten las mismas reglas entre links y contact. Usan color: `var(--muted)` para no competir con el contenido principal. El hover cambia a `var(--primary)` para dar retroalimentación visual. La propiedad `transition: color var(--transition)` hace el cambio suave.

La barra inferior `site-footer__bottom` usa `display: flex` con `justify-content: space-between` para alinear copyright a la izquierda y etiqueta a la derecha. El `border-top` y `padding-top` crean la separación visual. El `font-size: 13px` y color: `var(--muted)` mantienen la jerarquía: esta información es secundaria.

En el tema claro, los colores se adaptan automáticamente porque todas las reglas usan variables CSS: `var(--border)`, `var(--text)`, `var(--muted)`. No se necesita CSS adicional para el tema claro.

styles.css

```
.site-footer {
  margin-top: 40px;
  padding: 48px 32px 24px;
  background: rgba(255, 255, 255, 0.04);
  border-top: 1px solid var(--border);
}
.site-footer__grid {
  display: grid;
  grid-template-columns: 2fr 1fr 1fr;
  gap: 40px;
  max-width: 1120px;
  margin: 0 auto 32px;
}
.site-footer__brand .brand__mark {
  display: inline-flex;
  align-items: center;
  justify-content: center;
  width: 36px;
  height: 36px;
  border-radius: 50%;
  background: linear-gradient(135deg, var(--primary), var(--secondary));
  color: #ffffff;
  font-weight: 800;
  font-size: 14px;
  margin-right: 10px;
}
.site-footer__brand strong {
  font-size: 18px;
  font-weight: 700;
}
.site-footer__brand p {
  margin-top: 10px;
  font-size: 14px;
  max-width: 320px;
}
.site-footer__links strong,
.site-footer__contact strong {
```

```

    display: block;
    font-size: 14px;
    font-weight: 700;
    text-transform: uppercase;
    letter-spacing: 0.04em;
    margin-bottom: 14px;
    color: var(--text);
  }
  .site-footer__links nav {
    display: grid;
    gap: 8px;
  }
  .site-footer__links a,
  .site-footer__contact a {
    display: block;
    font-size: 14px;
    color: var(--muted);
    text-decoration: none;
    transition: color var(--transition);
  }
  .site-footer__links a:hover,
  .site-footer__contact a:hover {
    color: var(--primary);
  }
  .site-footer__bottom {
    display: flex;
    justify-content: space-between;
    align-items: center;
    gap: 16px;
    max-width: 1120px;
    margin: 0 auto;
    padding-top: 24px;
    border-top: 1px solid var(--border);
    font-size: 13px;
    color: var(--muted);
    flex-wrap: wrap;
  }

```

JavaScript

Este componente no requiere JavaScript propio. Es completamente estatico: HTML y CSS son suficientes para todo su funcionamiento. Los enlaces internos usan hash (#inicio, #componentes) que el navegador maneja automaticamente. Si tu proyecto usa el Router de secciones, los enlaces con data-section se conectan automaticamente a la clase RouterSections.

Si quieres agregar comportamiento dinamico, puedes hacerlo opcionalmente:

Actualizar el año automaticamente: reemplaza 2026 con `new Date().getFullYear()`.

Abrir enlaces externos en nueva pestana: ya esta resuelto con `target=_blank`.

Scroll suave al hacer clic en enlaces internos: agrega `scroll-behavior: smooth` al html.

Errores comunes y como solucionarlos

El footer tapa contenido en movil: si el margin-top del footer es muy pequeno o si el contenido anterior tiene margin-bottom negativo, el footer puede quedar pegado. Asegurate de que el elemento anterior tenga suficiente margin-bottom o que el footer tenga margin-top adecuado.

Los enlaces no funcionan: si usas rutas con hash (#inicio) pero tu pagina usa un router SPA que intercepta hashchange, el navegador puede no hacer scroll. Conecta los enlaces con data-section al router o usa scrollToView manualmente.

El footer no llega al final: si el body es mas corto que la ventana, el footer queda en medio de la pantalla. Solucion: usa min-height: 100vh en el contenedor principal o usa flexbox en el body para empujar el footer al final.

Texto del copyright desborda: si el texto del copyright es muy largo y la pantalla es estrecha, los dos span pueden quedar en la misma linea y desbordarse. La propiedad flex-wrap: wrap en site-footer__bottom lo soluciona: si no caben, se apilan.

Checklist de prueba antes de reutilizar

El footer se ve al final de la pagina sin tapar contenido.

Los enlaces internos llevan a las secciones correctas.

Los enlaces externos abren en nueva pestana y tienen rel=noreferrer.

El color y el contraste siguen siendo legibles en tema oscuro y claro.

El footer se adapta a tres columnas en escritorio, dos en tablet y una en movil.

No hay scroll horizontal en ninguna resolucion.

El texto del copyright se lee bien y no desborda.

Que se puede reutilizar directamente

El patron de grid de tres columnas con 2fr 1fr 1fr se puede adaptar para cualquier seccion de informacion: equipo, servicios, caracteristicas, testimonios o contacto.

La barra inferior con display: flex, justify-content: space-between y border-top se puede reutilizar para cualquier separador de contenido: copyright, terminos, version, estado o creditos.

El sistema de enlaces con hover que cambia de color usando var(--primary) es un patron universal que funciona en cualquier lista de navegacion.

El uso de variables CSS para colores, bordes y espacios permite que el footer se adapte automaticamente a cualquier tema sin escribir CSS adicional.

Variables, clases, ids y atributos que debes cambiar

- **Elemento** (Para que sirve): Que debes cambiar
- **siteFooter** (id del contenedor principal): Cambia si ya existe ese id
- **site-footer** (clase del contenedor principal): Cambia si colisiona con otra clase
- **site-footer__grid** clase del grid de columnas: Cambia si colisiona con otra clase
- **site-footer__brand** clase de la columna de marca: Cambia si colisiona con otra clase

- **site-footer__links** clase de la columna de enlaces: Cambia si colisiona con otra clase
- **site-footer__contact** clase de la columna de contacto: Cambia si colisiona con otra clase
- **site-footer__bottom** clase de la barra inferior: Cambia si colisiona con otra clase
- **brand__mark** (clase del logo circular): No cambies porque ya existe en el header
- **CW** (iniciales del logo): Cambia por las iniciales de tu proyecto
- **Componentes Web** (nombre del proyecto): Cambia por el nombre real
- **Banco de...** (descripcion de la marca): Cambia por la descripcion real
- **Secciones** (titulo de la columna de enlaces): Cambia por el titulo que necesites
- **Inicio** (texto del enlace interno): Cambia por las secciones de tu proyecto
- **Contacto** (titulo de la columna de contacto): Cambia por el titulo que necesites
- **Email / GitHub** (textos de los enlaces de contacto): Cambia por tus redes sociales reales
- **mailto** (href del correo): Cambia por tu correo real
- **https://github** (href de GitHub): Cambia por tu URL real
- **2026 Carlos Madero** (texto del copyright): Cambia por tu nombre y año
- **55 funcionalidades** (etiqueta informativa): Cambia por el dato de tu proyecto

56. Tarjetas de precios Bootstrap

Que hace?

Es una actividad guiada para construir una seccion de tarifas o precios responsiva usando la logica de componentes de Bootstrap como referencia, pero adaptada al estilo visual de este proyecto. Cuenta con tres tarjetas de plan (Basico, Popular, Equipo) donde se resalta el plan Pro mediante bordes, sombras y degradados distintivos. Todo adaptado para pantallas moviles sin romperse.

Objetivo de la actividad

Aprender a maquetar multiples tarjetas alineadas en rejilla responsiva (Grid), implementar badges de categoria y botones de accion, diferenciar visualmente un elemento destacado (featured card) y utilizar variables de CSS para que el diseno se adapte a temas claro y oscuro de manera limpia.

Estructura general del ejercicio

El contenedor principal con clase pricing-lab agrupa las tarjetas. Cada tarjeta es un article con clase pricing-lab__card. La tarjeta destacada tiene la clase adicional pricing-lab__card--featured. Cada tarjeta contiene un badge pricing-lab__badge, un titulo de plan (h4), un precio pricing-lab__price y una lista de características pricing-lab__list.

HTML minimo que debes copiar

index.html

```
<div class="pricing-lab">
  <article class="pricing-lab__card">
    <span class="pricing-lab__badge">Basico</span>
    <h4>Starter</h4>
    <p class="pricing-lab__price">$19<span>/mes</span></p>
    <ul class="pricing-lab__list">
      <li>1 proyecto</li>
      <li>Soporte por correo</li>
      <li>Componentes base</li>
    </ul>
    <button type="button" class="secondary-button">Elegir plan</button>
  </article>

  <article class="pricing-lab__card pricing-lab__card--featured">
    <span class="pricing-lab__badge">Popular</span>
    <h4>Pro</h4>
    <p class="pricing-lab__price">$49<span>/mes</span></p>
    <ul class="pricing-lab__list">
      <li>10 proyectos</li>
      <li>Soporte prioritario</li>
      <li>Layouts premium</li>
    </ul>
    <button type="button" class="primary-button">Elegir plan</button>
  </article>

  <article class="pricing-lab__card">
    <span class="pricing-lab__badge">Equipo</span>
```

```

    <h4>Scale</h4>
    <p class="pricing-lab__price">$99<span>/mes</span></p>
    <ul class="pricing-lab__list">
      <li>Proyectos ilimitados</li>
      <li>Equipo colaborativo</li>
      <li>Soporte dedicado</li>
    </ul>
    <button type="button" class="secondary-button">Elegir plan</button>
  </article>
</div>

```

CSS del componente

styles.css

```

.pricing-lab {
  display: grid;
  grid-template-columns: repeat(auto-fit, minmax(220px, 1fr));
  gap: 16px;
}
.pricing-lab__card {
  display: grid;
  gap: 14px;
  padding: 20px;
  border: 1px solid var(--border);
  border-radius: var(--radius-md);
  background: rgba(255, 255, 255, 0.06);
}
.pricing-lab__card--featured {
  background: linear-gradient(180deg, rgba(37, 99, 235, 0.2), rgba(255, 255, 255,
0.08));
  border-color: rgba(37, 99, 235, 0.5);
  box-shadow: 0 18px 36px rgba(37, 99, 235, 0.16);
}
[data-theme="light"] .pricing-lab__card {
  background: rgba(255, 255, 255, 0.82);
}
.pricing-lab__badge {
  display: inline-flex;
  width: fit-content;
  padding: 6px 10px;
  border-radius: 999px;
  background: rgba(37, 99, 235, 0.16);
  color: #bfdbfe;
  font-size: 12px;
  font-weight: 700;
}
[data-theme="light"] .pricing-lab__badge {
  color: #1d4ed8;
  background: rgba(37, 99, 235, 0.1);
}
.pricing-lab__price {
  margin: 0;
  font-size: 2rem;
  font-weight: 800;
  color: var(--text);
}

```

```

.pricing-lab__price span {
  font-size: 0.9rem;
  font-weight: 600;
  color: var(--muted);
}
.pricing-lab__list {
  display: grid;
  gap: 8px;
  margin: 0;
  padding-left: 18px;
  color: var(--muted);
}

```

JavaScript

No requiere JS. Esta actividad es puramente de maquetación HTML y CSS. Si deseas agregar interacción, puedes implementar un toggle de facturación mensual/anual o animar la tarjeta al pasar el cursor (hover).

Variables, clases, ids y atributos que debes cambiar

- ****Elemento**** (Para que sirve): Que debes cambiar
- ****pricing-lab**** (clase del contenedor principal): Cambia si colisiona con otra clase
- ****pricing-lab__card** clase de cada tarjeta de plan**: Cambia si colisiona con otra clase
- ****pricing-lab__card--featured** clase para destacar el plan Pro**: Cambia el degradado y sombra si deseas otro estilo
- ****pricing-lab__badge** clase del badge del plan**: Cambia los colores de fondo y texto si usas otra paleta
- ****pricing-lab__price** clase del precio**: Ajusta el tamaño de fuente si es necesario
- ****pricing-lab__list** clase de la lista de beneficios**: Cambia el espaciado o agrega iconos tipo check

Paso a paso para implementarlo en otro proyecto

1. Copia la estructura HTML del contenedor pricing-lab en la sección correspondiente de tu index.html.
2. Copia los estilos CSS de pricing-lab, pricing-lab__card, featured, badge, price y list en tu styles.css.
3. Asegurate de tener configuradas las variables de tema (claro/oscuro) en tu :root o cambialas por colores planos fijos si tu proyecto no tiene temas dinámicos.
4. Ajusta la anchura mínima en el minmax de grid-template-columns según las dimensiones deseadas para tus tarjetas.
5. Cambia el contenido de los planes (nombres, precios, características) por los de tu propio servicio.

Errores comunes y como solucionarlos

- Las tarjetas se ven amontonadas en pantallas medianas: el breakpoint o el tamaño mínimo en minmax(220px, 1fr) es demasiado grande o pequeño. Ajusta el valor del minmax.
- El plan destacado no resalta lo suficiente en tema claro: ajusta el border-color y el

background para el selector [data-theme="light"] .pricing-lab__card--featured.

- La lista de beneficios tiene vietas por defecto (puntos): el navegador aplica estilos de lista por defecto. El CSS incluye reset o define list-style: square/disc si deseas conservarlos, o puedes quitarlos con list-style: none y padding: 0.

Checklist de prueba antes de reutilizar

- El layout es responsive y pasa de 3 columnas en escritorio a 1 columna en pantallas de movil.
- El plan destacado Pro es claramente distinguible de los demas planes.
- Los textos y precios son legibles en tema claro y oscuro.
- No hay scroll horizontal en resoluciones pequenas.

57. Barra de desplazamiento personalizada

Que hace

Aplica estilos personalizados y modernos a las barras de desplazamiento (scrollbars) de la pagina o de contenedores especificos. Reemplaza la barra nativa del sistema operativo por una barra mas delgada, elegante y con transiciones.

Donde se usa en industria

Paneles de control, sidebars de navegacion, bloques de codigo, tablas con scroll horizontal, areas de texto largas, chats y dashboards premium.

Como se personaliza

Ajusta el width (ancho), los border-radius (redondeado), el background-color del track (pista) y el thumb (deslizador). Puedes usar degradados y efectos hover.

Mapa rapido de reciclaje

1

Ubica el contenedor con desbordamiento (overflow: auto o scroll) que quieres estilizar.

2

Copia los selectores CSS especificos de scrollbar.

3

Ajusta las variables de color y el ancho segun el diseno de tu interfaz.

4

Prueba tanto en navegadores basados en Chromium (Chrome, Edge, Opera, Safari) como en Firefox.

HTML minimo que debes copiar

index.html

```
<div class="custom-scrollable-container">
  <div class="custom-scrollable-content">
    <p>Contenido largo que genera scroll...</p>
  </div>
</div>
```

CSS del componente

styles.css

```
/* Contenedor de la demo */
.custom-scrollable-container {
  border: 1px solid var(--border);
  border-radius: var(--radius-md);
  background: rgba(255, 255, 255, 0.03);
  padding: 16px;
}
.custom-scrollable-content {
  max-height: 160px;
  overflow-y: auto;
  padding-right: 8px;
```

```

    /* Soporte para Firefox (estandar moderno) */
    scrollbar-width: thin;
    scrollbar-color: rgba(255, 255, 255, 0.18) rgba(255, 255, 255, 0.02);
}

/* Soporte para Webkit (Chrome, Edge, Safari) */
.custom-scrollable-content::-webkit-scrollbar {
    width: 6px;
    height: 6px;
}
.custom-scrollable-content::-webkit-scrollbar-track {
    background: rgba(255, 255, 255, 0.02);
    border-radius: 999px;
}
.custom-scrollable-content::-webkit-scrollbar-thumb {
    background: rgba(255, 255, 255, 0.18);
    border-radius: 999px;
}
.custom-scrollable-content::-webkit-scrollbar-thumb:hover {
    background: linear-gradient(180deg, var(--primary), var(--secondary));
}

```

JavaScript modular con export/import

funciones.js - no requiere JS

// No requiere JS. Estilo de scroll puro con CSS.

JavaScript normal sin modulos

script-normal.js - no requiere JS

// No requiere JS. Estilo de scroll puro con CSS.

Variables, clases, ids y atributos que debes cambiar

Elemento

Para que sirve

Que debes cambiar

.custom-scrollable-content

Contenedor con scroll.

Reemplaza por tu selector, ej. .guide-sidebar o pre.

width / height

Dimensiones de la barra.

Cambia 6px a 8px o 4px segun el grosor deseado.

scrollbar-color

Estilos en Firefox.

El primer color es el thumb, el segundo es el track.

::-webkit-scrollbar-thumb

Deslizador en Chrome/Safari.

Cambia el color, agrega border o usa degradados en hover.

`::-webkit-scrollbar-track`

Pista de fondo de la barra.

Puedes poner un color solido o mantenerlo transparente.

Implementacion paso a paso

1

Identifica el contenedor en tu HTML que tiene scroll activo (por ejemplo, con `overflow-y: auto`).

2

Copia las propiedades de Firefox (`scrollbar-width` y `scrollbar-color`) dentro del selector de tu contenedor.

3

Copia las pseudoclases `::-webkit-scrollbar` para definir el ancho/alto de la barra en Chrome/Edge.

4

Copia `::-webkit-scrollbar-track` para el fondo y `::-webkit-scrollbar-thumb` para la barra que se mueve.

5

Personaliza los colores en estado normal y hover para que hagan juego con tu paleta visual.

6

Prueba la interaccion desplazando el contenido y pasando el mouse por encima de la barra.

Errores comunes y como solucionarlos

La barra no aparece personalizada: revisa que el contenedor tenga `overflow-y: auto` o `overflow-x: auto`. Si no hay desbordamiento de contenido, la barra no se muestra.

Diferencia entre Firefox y Chrome: Firefox no soporta `::-webkit-scrollbar`, usa `scrollbar-width` y `scrollbar-color`. Asegurate de incluir ambas soluciones.

`Scrollbar-color` no funciona: en Firefox, el valor debe ser de dos colores validos, ej: `scrollbar-color: thumb track`. No uses gradientes directos en esta propiedad.

El contenido se superpone a la barra: agrega un `padding-right` al contenedor scrollable para dejar espacio a la barra y evitar que el texto quede pegado.

Checklist de prueba antes de reutilizar

La barra de desplazamiento aparece mas delgada y estilizada que la nativa.

La barra reacciona al hover cambiando de color.

Funciona tanto en scroll vertical como en scroll horizontal (ej. bloques de codigo pre).

No hay desbordamientos raros de disenio en movil.

Es legible y visible tanto en tema claro como en tema oscuro.

Mejoras opcionales

Usa variables CSS globales para que todas las barras de tu proyecto cambien de color al

mismo tiempo.

Agrega un efecto de borde transparente al thumb usando background-clip: padding-box para simular espacio flotante.

Conecta con JS para ocultar la barra cuando el usuario no esta moviendo el cursor y mostrarla solo en scroll.

Aplica el scrollbar personalizado al body de la pagina principal (html, body) si quieres un look 100% homoganeo.